

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology

FINAL PROJECT REPORT

Course: Introduction to Software Engineering

Virtual Stock Market Simulation and Consultancy Platform

System name: SoictStock

Group: *To be filled*

Class code: *To be filled*

Instructor: *To be filled*

Hanoi, June 15, 2026

Member Assignment

This section records the planned division of work for the project team. The roles should be updated with the actual names and student IDs before submission.

Table 1: Member assignment

Full name	Student ID	Main work
Member 1	To be filled	Team lead, requirement analysis, project planning, report integration, final presentation.
Member 2	To be filled	Backend API, MongoDB connection, authentication, portfolio and trading logic.
Member 3	To be filled	Frontend interface, simulation dashboard, portfolio page, responsive layout.
Member 4	To be filled	Market simulation engine, WebSocket price streaming, scenarios, stock data and order book.
Member 5	To be filled	Learning center, quizzes, badges, chatbot, advisor content, blog and community module.
Member 6	To be filled	Testing, deployment, Docker setup, user guide, screenshots and demo validation.

Contents

Member Assignment	i
1 Problem Survey	1
1.1 Problem Description	1
1.2 Problem Survey and Existing Practices	1
1.3 Client, Users, and Stakeholders	3
1.4 Project Scope	3
1.4.1 In Scope	3
1.4.2 Out of Scope	4
1.5 Basic Business Processes	4
1.6 Business Function Decomposition	6
2 Software Requirements Specification	8
2.1 General Introduction	8
2.1.1 Purpose	8
2.1.2 Definitions	9
2.2 System Actors	9
2.3 Main Use Cases	10
2.3.1 Guest Use Cases	10
2.3.2 Registered User Use Cases	10
2.3.3 Simulation Engine Use Cases	10
2.3.4 Initialization of main Use Cases	11
2.4 Use Case Diagrams	12
2.5 Use Case Specifications	16
2.5.1 UC-01: Create blog	16
2.5.2 UC-02: Verify email	17
2.5.3 UC-03: Sign up / Sign in	17
2.5.4 UC-04: View market	18
2.5.5 UC-05: Trade stock	18
2.5.6 UC-06: View portfolio	18
2.5.7 UC-07: View leaderboard	19

2.5.8	UC-08: Learn lesson	19
2.5.9	UC-09: Take quiz	19
2.5.10	UC-10: Join contest	20
2.5.11	UC-11: Ask chatbot	20
2.5.12	UC-12: Manage market scenario	21
2.6	Functional Requirements	21
2.6.1	FR-01 Authentication and User Profile	21
2.6.2	FR-02 Market Simulation	21
2.6.3	FR-03 Trading and Order Management	22
2.6.4	FR-04 Portfolio Management and Risk Analytics	22
2.6.5	FR-05 Learning Center	22
2.6.6	FR-06 Chatbot, Advisor, Blog, Contest, and Leaderboard	23
2.7	Non-functional Requirements	24
3	Requirement Analysis	25
3.1	Identifying Analysis Classes	25
3.1.1	Responsibilities of Key Control Classes	30
3.1.2	BCE Analysis Diagram	31
3.2	Main Analysis Scenarios	35
3.3	Sequence Diagrams	36
3.3.1	User Registration	37
3.3.2	Real-Time Market Streaming	37
3.3.3	Market Trade Execution	38
3.3.4	Pending Order Lifecycle	38
3.4	Logical Data Model	39
3.5	Data Dictionary	39
3.6	Implementation Scope and Consistency Notes	44
4	System Design	45
4.1	Selected Architecture	45
4.2	Main Components	46
4.3	Component Diagram	49
4.4	Database Design	50
4.4.1	Main Collections	50
4.4.2	Collection Relationship Diagram	52
4.4.3	Index Design	52
4.5	Detailed Package Design	53
4.5.1	Backend Package Design	53
4.5.2	Frontend Package Design	54
4.6	Runtime Interaction Design	55

4.6.1	Application Startup and Market Streaming	55
4.6.2	Trading and Portfolio Update Flow	55
4.6.3	Pending Order Flow	56
4.7	API Design	56
4.8	Market Simulation Design	59
4.9	Interface Design	59
4.10	Deployment Design	61
4.11	Security and Data Integrity Design	61
4.12	Design Consistency Notes	62
4.13	Summary	63
5	Demo Program Implementation	64
5.1	Source Code Organization	64
5.2	Tools and Libraries	66
5.3	Backend Implementation	66
5.3.1	Application Startup	66
5.3.2	Database and Persistence	67
5.3.3	Authentication	67
5.3.4	Market Simulation and Price Streaming	68
5.3.5	Trading, Orders, Portfolio, and Leaderboard	68
5.3.6	News, Scenarios, and Advisor	69
5.3.7	Learning Center	69
5.3.8	Chatbot	69
5.3.9	Blog and Community	70
5.3.10	Contest Module	71
5.4	Frontend Navigation and Page Implementation	71
5.5	Deployment Implementation	72
5.6	Demo Interface Screenshots	73
6	System Testing	74
6.1	Testing Strategy	74
6.2	Functional Test Cases	74
6.3	Non-functional Testing	77
6.4	Requirement Traceability Matrix	77
7	Installation and User Guide	79
7.1	System Requirements	79
7.1.1	Hardware Requirements	79
7.1.2	Software Requirements	79
7.2	Environment Configuration	79

7.3	Docker Installation	80
7.3.1	Start the Application	80
7.3.2	View Logs	80
7.3.3	Stop the Application	81
7.4	Manual Backend Setup	81
7.5	Manual Frontend Setup	81
7.6	First Launch Checklist	82
7.7	User Guide	82
7.7.1	Account and Profile	82
7.7.2	Trading Simulator	82
7.7.3	Portfolio	82
7.7.4	Learning Center	83
7.7.5	Chatbot	83
7.7.6	Blogs and Community Posts	83
7.7.7	Writing and Managing Blog Posts	84
7.7.8	Voting and Commenting on Blogs	84
7.7.9	Contest Mode	84
7.7.10	Leaderboard	85
7.8	Troubleshooting	85
8	Project Management and Configuration Management	86
8.1	Development Methodology	86
8.2	Work Breakdown Structure	87
8.3	Risk Management	87
8.4	Configuration Management	87
8.4.1	Configuration Items	88
8.4.2	Git Strategy	88
8.4.3	Change Management Workflow	88
9	Evaluation and Future Work	89
9.1	System Evaluation	89
9.1.1	Strengths	89
9.1.2	Limitations	89
9.2	Future Work	90
10	Conclusion	92
A	API Details	93
B	Sample Data	94

C Full Test Cases	95
D Screenshots	96

List of Figures

1.1	Business function decomposition of SoictStock	6
2.1	Overall Use Case Diagram	13
2.2	Level-2 Use Case Diagram — Trading and Portfolio	14
2.3	Level-2 Use Case Diagram — Learning, Chatbot, and Blog	15
2.4	Level-2 Use Case Diagram — Contest and Leaderboard	16
3.1	Boundary–Control–Entity analysis diagram for SoictStock	31
3.2	Boundary–Control–Entity analysis diagram for SoictStock	32
3.3	Boundary–Control–Entity analysis diagram for SoictStock	33
3.4	Boundary–Control–Entity analysis diagram for SoictStock	34
3.5	Boundary–Control–Entity analysis diagram for SoictStock	35
3.6	Sequence diagram for user registration	37
3.7	Sequence diagram for real-time market streaming	37
3.8	Sequence diagram for executing a market trade	38
3.9	Sequence diagram for a pending order	38
3.10	ERD-style logical data model for MongoDB collections	39
4.1	Implementation-level architecture of SoictStock	46
4.2	Runtime component diagram of SoictStock	50
4.3	Logical MongoDB collection relationship diagram	52
4.4	Backend package organization	53
4.5	Frontend package organization	54
4.6	Runtime flow for backend startup and real-time market streaming . . .	55
4.7	Trade execution and portfolio update flow	55
4.8	Docker-based deployment topology	61
5.1	Placeholder for simulation dashboard screenshot	73
8.1	Work breakdown structure	87
8.2	Change management workflow	88

List of Tables

1	Member assignment	i
1.1	Client, users, and stakeholders	3
1.2	Business Process 1: User registration and portfolio initialization	4
1.3	Business Process 2: Real-time market simulation	4
1.4	Business Process 3: Virtual trading	5
1.5	Business Process 4: Portfolio and risk analytics	5
1.6	Business Process 5: Learning and quiz progress	5
1.7	Business Process 6: Chatbot and advisor support	5
1.8	Business Process 7: Blog and community discussion	6
1.9	Business Process 8: Contest and leaderboard	6
1.10	Business function decomposition table	7
2.1	Glossary	9
2.2	System actors	9
2.3	SoictStock System Use Cases Specification	11
2.4	Non-functional Requirements	24
3.1	Main Boundary Classes	25
3.2	Main Control Classes	26
3.3	Main Entity Classes	28
3.4	Core user, portfolio, and trading data	40
3.5	Market, analytics, and content data	41
3.6	Learning, chatbot, blog, and contest data	42
4.1	Main design components and implementation mapping	47
4.2	MongoDB collections used by the implementation	50
4.3	Main REST and WebSocket API design	56
4.4	Main user interfaces and design purpose	60
5.1	Main source code folders	65
5.2	Technology stack used in implementation	66
5.3	Frontend routes	72

6.1	Functional test cases	74
6.1	Functional test cases	75
6.1	Functional test cases	76
6.2	Non-functional testing plan and status	77
6.3	Requirement traceability matrix	77
6.3	Requirement traceability matrix	78
7.1	Environment file usage	80
7.2	Common setup problems	85
8.1	Suggested sprint plan	86
8.2	Risk assessment and mitigation	87

Chapter 1

Problem Survey

1.1 Problem Description

The proposed project, SoictStock, is a web-based virtual stock market simulation and financial education platform. The system is designed for users who want to understand how equity trading works without using real money. Many beginners find concepts such as price movement, market volatility, order types, portfolio allocation, profit and loss, drawdown, and risk management difficult to understand through theory alone. Real trading platforms are not suitable for early learning because they expose inexperienced users to financial risk.

SoictStock addresses this problem by providing a risk-free environment where users can observe simulated stock prices, place buy and sell orders, manage a virtual portfolio, join trading contests, learn financial concepts through lessons and quizzes, and ask a chatbot for educational guidance. The system also introduces SoictStock's consultancy direction through advisor profiles, educational blog posts, market insights, and a mock advisor/backtesting service.

The system is strictly intended for education and simulation. It does not support real-money trading, brokerage integration, or legally binding financial advice. All transactions, market prices, consultant profiles, rewards, and advisory responses are simulated.

1.2 Problem Survey and Existing Practices

Current financial learning methods often fall into three groups. First, traditional lessons and articles explain stock market concepts but provide limited interaction. Second, real trading platforms provide practical experience but require real money and may lead to losses. Third, simple stock market games allow simulated trading but often lack structured learning, realistic market behavior, portfolio analytics, and explanation of

risk.

The target users of SoictStock need an integrated learning environment that combines:

- real-time simulated stock prices;
- virtual buy/sell trading and order practice;
- portfolio analytics and risk metrics;
- lessons, quizzes, badges, and practice tasks;
- chatbot support for financial concepts and platform usage;
- blogs and community discussion;
- contests and leaderboards to increase engagement.

Therefore, the project is not only a trading simulator. It is an educational ecosystem that connects market simulation, learning, portfolio reflection, and community engagement.

1.3 Client, Users, and Stakeholders

Table 1.1: Client, users, and stakeholders

Role	Description
Client / Course evaluator	The instructor and teaching team who define the course requirements and evaluate the final software engineering report and demo.
Learner / Registered user	A user who signs up, receives virtual cash, trades simulated stocks, studies lessons, completes quizzes, and uses the chatbot.
Competitive user	A user who joins contests, trades with a contest portfolio, and competes on the leaderboard.
Blog author	A signed-in user who creates draft posts, publishes educational posts, archives posts, and interacts with comments and votes.
Platform maintainer	A developer or operator who maintains stock data, learning content, contests, deployment configuration, and service availability.
SoictStock consultant profile	A simulated professional profile used to present consultancy-oriented educational content.
Development team	The group responsible for requirements, design, implementation, testing, deployment, and documentation.
Stakeholders	Learners, instructors, the development team, and people interested in financial technology education.

1.4 Project Scope

1.4.1 In Scope

The implemented project scope includes:

- user sign-up, sign-in, and profile update;
- virtual wallet and default portfolio initialization;
- simulated stock list, quotes, and historical price charts;
- real-time price updates through WebSocket;
- buy/sell simulated trades and order management;
- portfolio value, holdings, realized and unrealized profit/loss;
- risk metrics such as volatility, Sharpe ratio, and maximum drawdown;

- leaderboard and trading contests;
- market scenarios such as crisis, technology bubble, pandemic shock, and inflation scenario;
- news display and news injection for market context;
- learning paths, lessons, quizzes, badges, market analysis lab, and pattern game;
- chatbot learning assistant and mock advisor/backtesting service;
- blog/community module with drafts, publishing, archiving, votes, and comments;
- Docker-based deployment support.

1.4.2 Out of Scope

The following features are not included in the current project version:

- real-money trading;
- real brokerage or stock exchange integration;
- legal financial advice;
- real consultant booking and payment;
- production-grade role-based admin dashboard;
- production-grade authentication session management;
- real contest rewards.

1.5 Basic Business Processes

This section summarizes the main business processes of SoictStock using an input–process–output structure.

Table 1.2: Business Process 1: User registration and portfolio initialization

Input	Process	Output
Email, password, display name	Validate account data, hash the password, create a user record, and initialize a default portfolio with virtual cash.	User account, hashed password, default portfolio, initial virtual cash balance.

Table 1.3: Business Process 2: Real-time market simulation

Input	Process	Output
-------	---------	--------

Stock list, base price, sector, drift, volatility, market regime	Generate historical bars and real-time ticks using the simulation engine. Apply jump events, momentum, mean reversion, volatility clustering, scenarios, and news impact.	Current prices, stock quotes, historical bars, real-time WebSocket price stream.
--	---	--

Table 1.4: Business Process 3: Virtual trading

Input	Process	Output
User ID, ticker, trade type, quantity, current price	Validate ticker, check virtual cash or holdings, execute buy/sell logic, update portfolio, record transaction, and update leaderboard.	Filled transaction, updated cash, updated holdings, updated leaderboard entry.

Table 1.5: Business Process 4: Portfolio and risk analytics

Input	Process	Output
Holdings, cash, transactions, price stream, portfolio snapshots	Calculate total value, realized/unrealized profit and loss, allocation, volatility, Sharpe ratio, and maximum drawdown.	Portfolio dashboard, allocation chart, transaction history, risk metrics.

Table 1.6: Business Process 5: Learning and quiz progress

Input	Process	Output
Lesson ID, section index, quiz answers, user ID	Mark completed sections, calculate quiz score, save quiz attempt, update progress and badges, and recommend next activities.	Learning progress, quiz result, badges, recommended lesson or task.

Table 1.7: Business Process 6: Chatbot and advisor support

Input	Process	Output
User message, market context, portfolio context, learning context	Detect intent, apply safety rules, generate an educational answer, suggest relevant pages or lessons, and store chat history.	Chatbot response, suggestion cards, educational disclaimer, saved history.

Table 1.8: Business Process 7: Blog and community discussion

Input	Process	Output
Blog title, content, stock tag, author ID	Create draft, update content, publish, archive, delete, vote, and comment on posts.	Published post, archived post, public profile posts, comments, votes.

Table 1.9: Business Process 8: Contest and leaderboard

Input	Process	Output
Contest ID, user ID, contest trade	Join active contest, initialize contest portfolio, restrict allowed tickers, execute contest trade, and rank participants.	Contest portfolio, contest transaction, contest leaderboard, reward/ranking display.

1.6 Business Function Decomposition

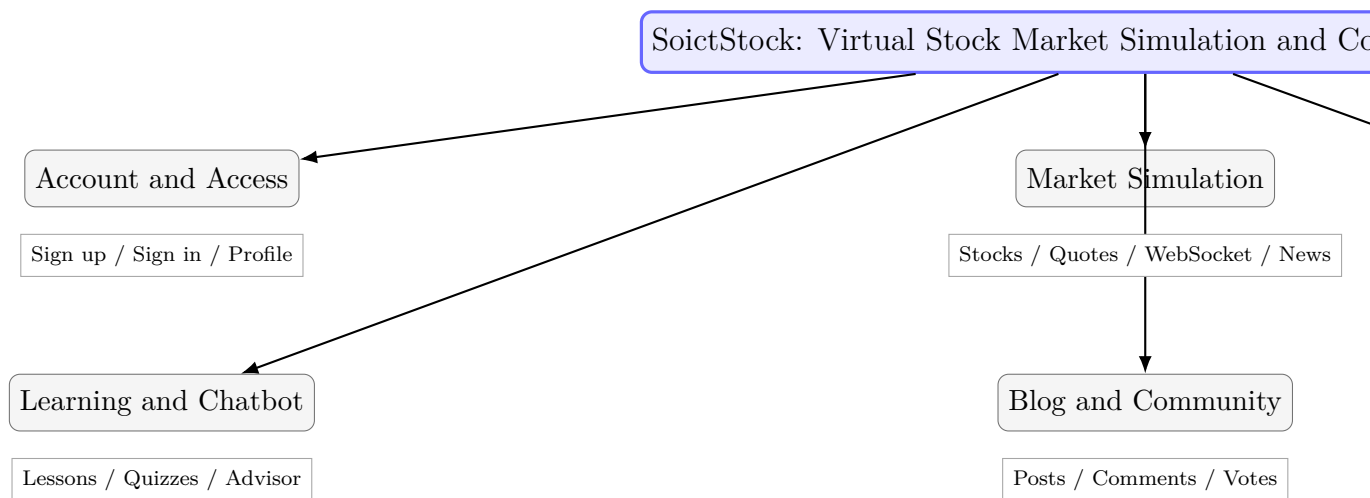


Figure 1.1: Business function decomposition of SoictStock

Table 1.10: Business function decomposition table

No.	Business function	Description	Status
1	Account and access management	Manage sign-up, sign-in, profile update, and default portfolio initialization.	Implemented
2	Market simulation management	Generate historical bars, real-time ticks, quotes, scenarios, and news-driven context.	Implemented
3	Trading management	Support direct buy/sell transactions, open orders, filled orders, and cancellation.	Implemented
4	Portfolio analytics	Calculate holdings, cash, portfolio value, profit/loss, allocation, volatility, Sharpe ratio, and maximum drawdown.	Implemented
5	Learning management	Manage lessons, quizzes, section completion, badges, learning progress, pattern game, and market analysis lab.	Implemented
6	Chatbot and advisor support	Provide educational Q&A, suggestion cards, safety disclaimers, mock advisor responses, and mock backtesting.	Implemented
7	Blog and community management	Manage draft posts, published posts, archived posts, comments, votes, and public user profiles.	Implemented
8	Contest and leaderboard management	Manage active contests, contest portfolios, contest trades, rewards, and rankings.	Implemented
9	Administration dashboard	A dedicated role-based admin dashboard for all platform data.	Future work

Chapter 2

Software Requirements Specification

2.1 General Introduction

This chapter specifies the requirements of SoictStock from the user's and system's perspective. The SRS defines the main actors, use cases, functional requirements, and non-functional requirements. It also provides a basis for system design and testing.

2.1.1 Purpose

The purpose of SoictStock is to help users learn stock market concepts through a realistic but risk-free simulation. The system combines trading practice, portfolio tracking, educational content, chatbot support, community discussion, and competitive learning.

2.1.2 Definitions

Table 2.1: Glossary

Term	Definition
Virtual stock	A simulated tradable asset representing a fictional or educational stock.
Virtual wallet	A simulated cash balance used for educational trading.
Portfolio	A collection of virtual cash, stock holdings, transactions, and performance metrics.
Order	A request to buy or sell a simulated stock.
Leverage position	A margin-based position (Long/Short) allowing users to amplify their simulated exposure.
Transaction	A completed trade that updates portfolio cash and holdings.
Tick	A generated price update in the simulated market.
Market scenario	A predefined market condition such as crisis or inflation that changes price behavior.
Learning path	A structured set of lessons, quizzes, and practice tasks.
Advisor	A simulated educational assistant, not a real financial advisor.

2.2 System Actors

Table 2.2: System actors

Actor	Description
Guest	A visitor who can view landing content, public blog posts, and sign-up/sign-in options.
Registered user	A user who can trade simulated stocks, view portfolio, learn lessons, take quizzes, create blogs, join contests, and ask the chatbot.
Contest participant	A registered user who joins a contest and trades with a contest-specific portfolio.
Blog author	A signed-in user who creates, updates, publishes, archives, or deletes their own blog posts.
Platform maintainer	A person who maintains the source code, seed data, deployment configuration, and future admin functions.

Simulation engine	An internal system component that generates historical prices and real-time ticks.
News injector	An internal service that injects market news into the simulation context.
Chatbot service	An internal service that generates educational responses and suggestion cards based on user messages and safety rules.

2.3 Main Use Cases

2.3.1 Guest Use Cases

- View landing page.
- Browse public blog posts.
- Sign up.
- Verify email
- Sign in.

2.3.2 Registered User Use Cases

- View market dashboard.
- View stock chart and quote.
- Place an order.
- Execute buy/sell trade.
- Cancel an open order.
- View portfolio, risk metrics, and transaction history.
- View leaderboard.
- Join contest and trade in contest arena.
- Read lessons, complete sections, and take quizzes.
- Use market analysis lab and pattern game.
- Ask chatbot.
- Create, update, publish, archive, and delete own blog posts.
- Vote and comment on published blog posts.

2.3.3 Simulation Engine Use Cases

- Generate historical price bars.
- Broadcast real-time price ticks.
- Apply market regimes and scenarios.
- Update current quotes.

2.3.4 Initialization of main Use Cases

Table 2.3: SoictStock System Use Cases Specification

No.	Code	Use case	Description	Actor
1	UC-01	Create blog	Create a new blog draft, edit content, publish, archive, or delete.	Blog Author
2	UC-02	Verify email	Verify the newly created account via an email link before signing in.	Guest
3	UC-03	Sign up / Sign in	Register a new account or log in to an existing account.	Guest
4	UC-04	View market	View market dashboard, stock list, stock quotes, and historical price charts.	Registered User
5	UC-05	Trade stock	Place orders, execute buy/sell trades, open/close leverage positions, and cancel open orders.	Registered User
6	UC-06	View portfolio	View cash balance, current holdings, transaction history, and risk metrics.	Registered User
7	UC-07	View leaderboard	View general platform leaderboard and see user rankings based on portfolio performance.	Registered User
8	UC-08	Learn lesson	View learning paths, read lesson content, and mark sections as completed.	Registered User
9	UC-09	Take quiz	Answer and submit quizzes related to learning paths and view scores.	Registered User
10	UC-10	Join contest	View active contests, join a contest, and trade in a contest-specific portfolio arena.	Contest Participant
11	UC-11	Ask chatbot	Ask educational questions about the platform, trading concepts, and risk.	Registered User
12	UC-12	Manage market scenario	Trigger predefined market scenarios such as crisis or inflation to affect simulation prices.	Platform Maintainer

2.4 Use Case Diagrams

This section includes one overall use case and three level-2 use cases. The diagrams below visually represent the main use cases of the system.

Diagram 1: Overall Use Case Diagram

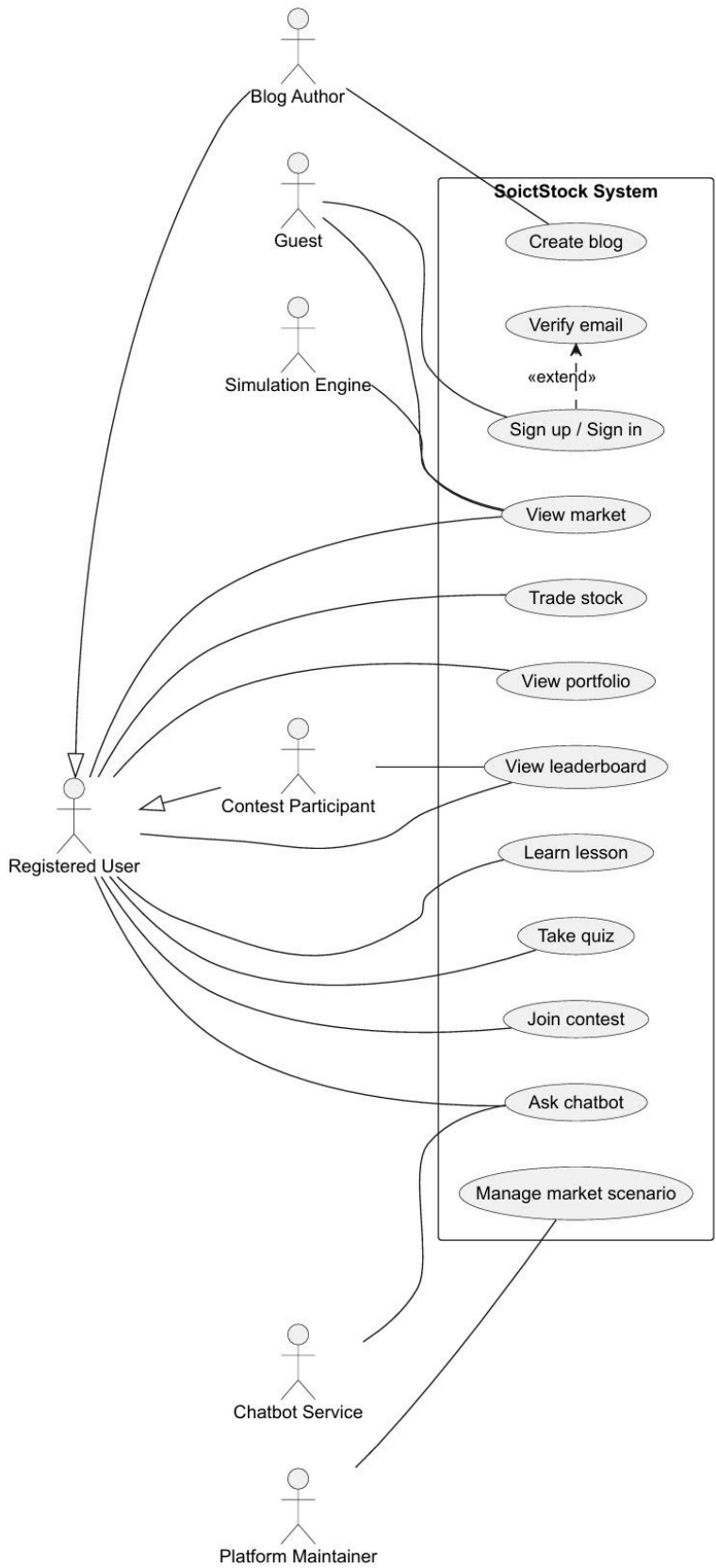


Figure 2.1: Overall Use Case Diagram

Diagram 2: Level-2 Use Case Diagram — Trading and Portfolio

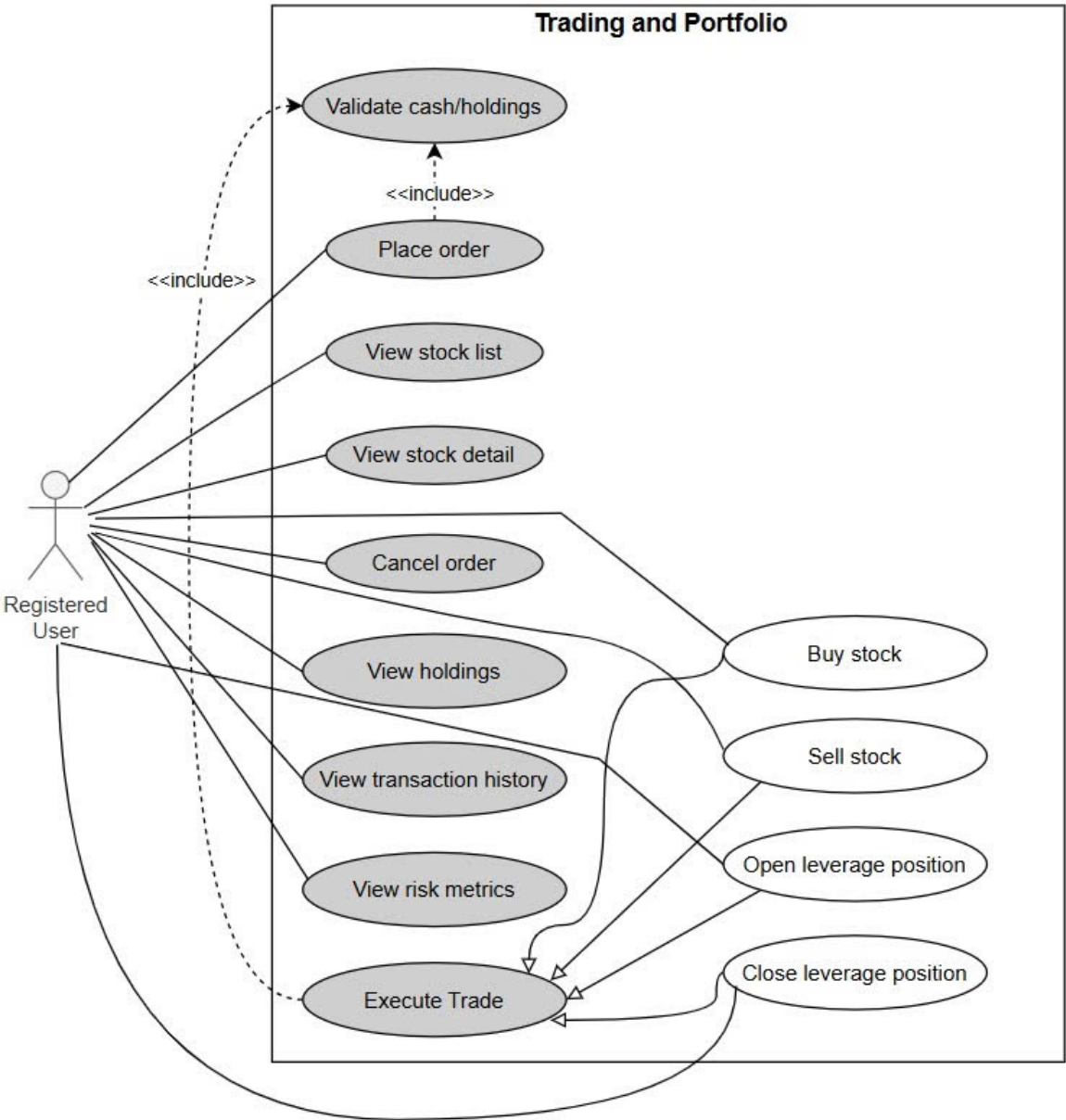


Figure 2.2: Level-2 Use Case Diagram — Trading and Portfolio

Diagram 3: Level-2 Use Case Diagram — Learning, Chatbot, and Blog

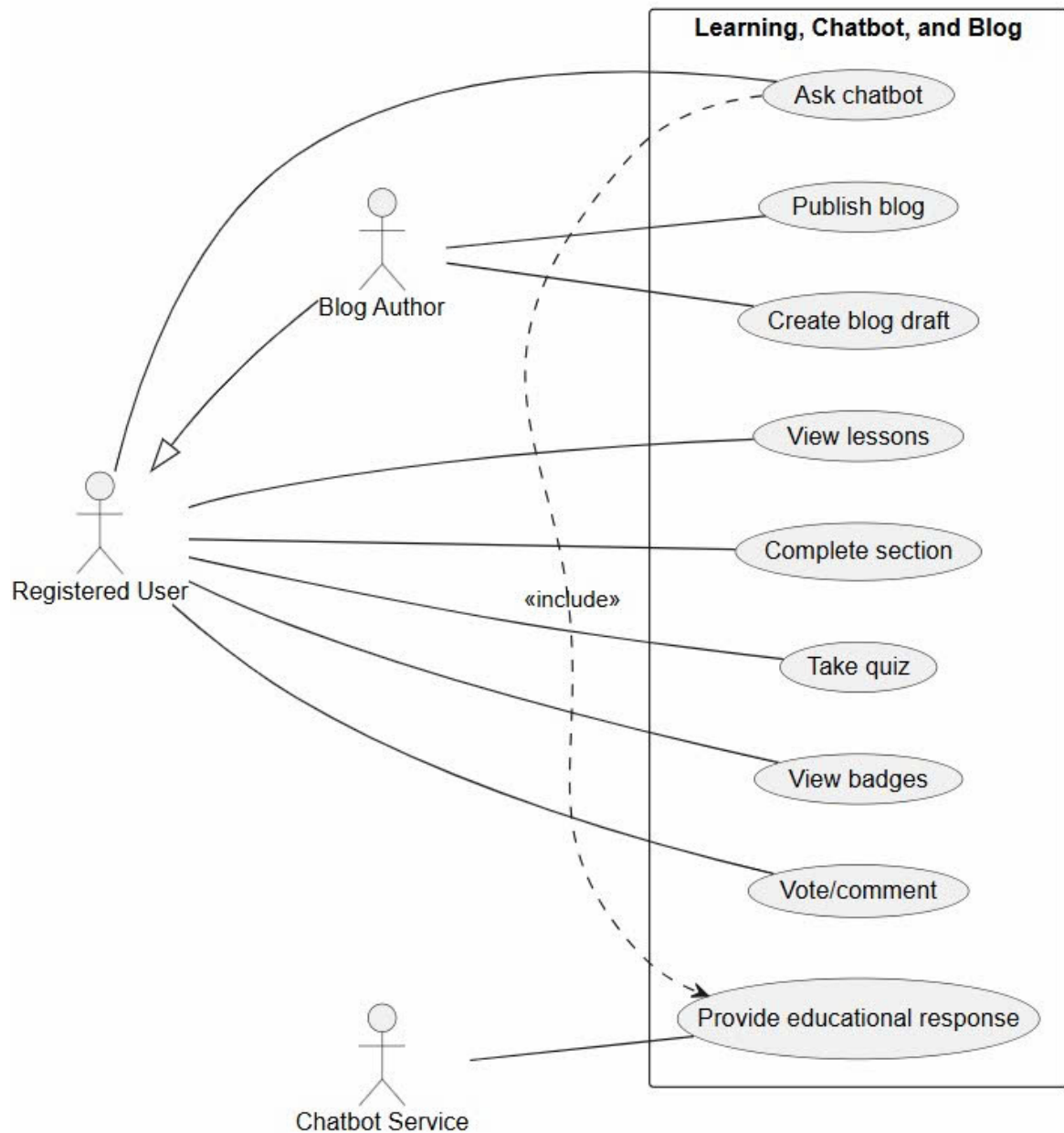


Figure 2.3: Level-2 Use Case Diagram — Learning, Chatbot, and Blog

Diagram 4: Level-2 Use Case Diagram — Contest and Leaderboard

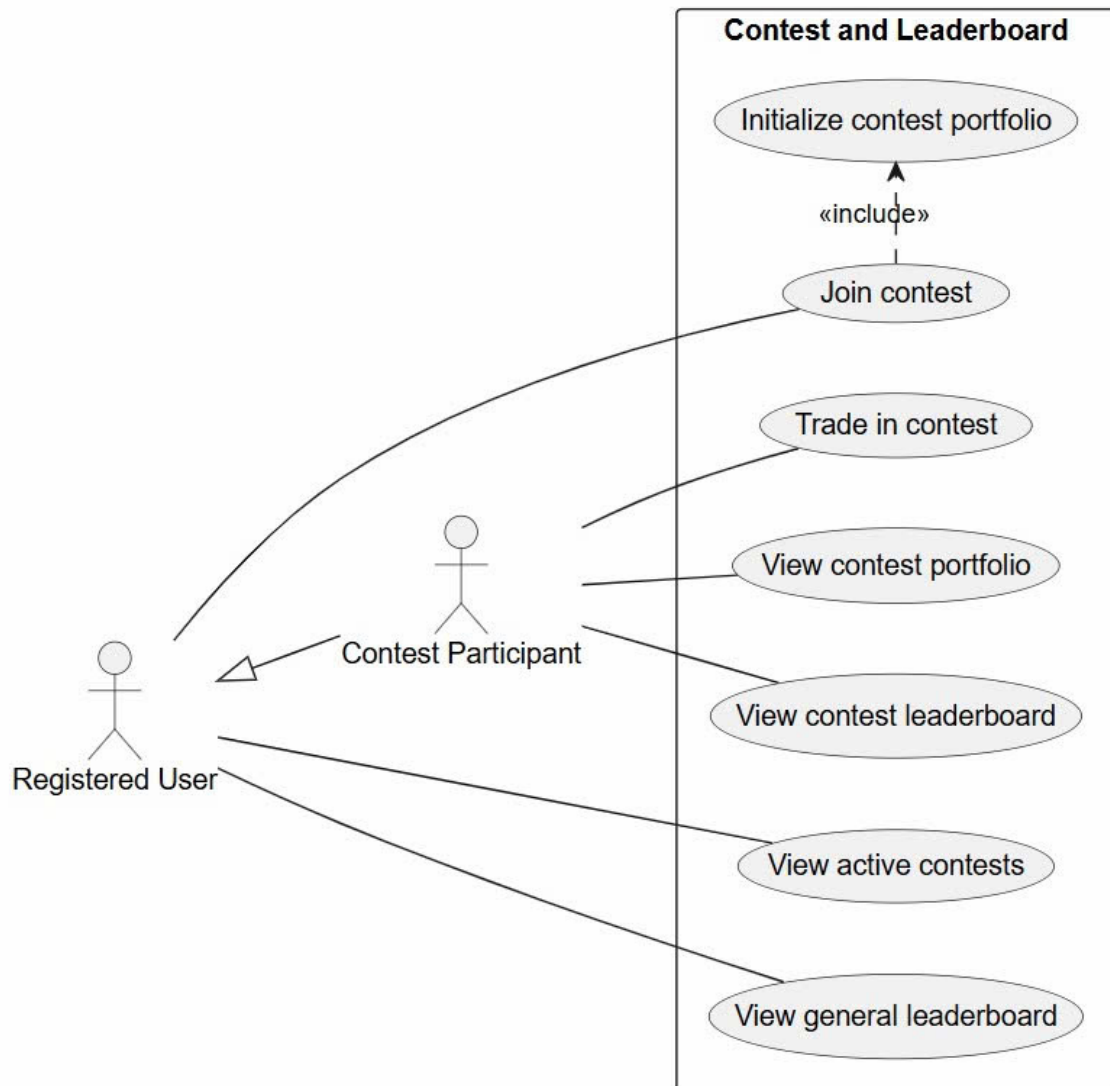


Figure 2.4: Level-2 Use Case Diagram — Contest and Leaderboard

2.5 Use Case Specifications

2.5.1 UC-01: Create blog

- **Actor:** Blog Author
- **Precondition:** The user is signed in and navigated to the blog management page.
- **Main flow:** The author creates a new draft, edits the title and content, and saves it. The author then chooses to publish the post. The system updates the

post status and makes it visible on the public blog page.

- **Alternative flow:** If the content is empty or invalid, the system displays a validation error. If the user does not have permission, the action is rejected.
- **Postcondition:** The blog post is created, stored, and appropriately visible based on its status (draft/published).

2.5.2 UC-02: Verify email

- **Actor:** Guest
- **Precondition:** The guest has successfully submitted the sign-up form but has not verified their email yet.
- **Main flow:** The user clicks the verification link sent to their email. The system validates the token. The system updates the user record to mark the email as verified. The system displays a success message allowing the user to sign in.
- **Alternative flow:** If the token is invalid or expired, the system returns an error message and prompts the user to request a new verification email.
- **Postcondition:** The user's account is verified and ready for sign in.

2.5.3 UC-03: Sign up / Sign in

- **Actor:** Guest
- **Precondition:** The visitor is on the landing page or authentication modal.
- **Main flow:** For sign-up, the guest enters email, password, and display name. The system hashes the password, creates a user record, and initializes a default portfolio with virtual cash. For sign-in, the guest enters their credentials. The system authenticates the user and establishes a session.
- **Alternative flow:** If the email already exists during sign-up, or if credentials are wrong during sign-in, the system displays an error message. If the email is unverified during sign-in, access is denied.
- **Postcondition:** A new account is created (sign-up), or the user is authenticated and logged into the platform (sign-in).

2.5.4 UC-04: View market

- **Actor:** Registered User
- **Precondition:** The user is signed in.
- **Main flow:** The user navigates to the market dashboard. The system displays a list of simulated stocks. The user clicks on a specific stock to view its detailed chart, historical prices, and current quotes updated in real-time via WebSocket.
- **Alternative flow:** If the WebSocket connection is lost, the frontend falls back to local simulation to ensure prices still visually update.
- **Postcondition:** The user observes real-time and historical market data successfully.

2.5.5 UC-05: Trade stock

- **Actor:** Registered User
- **Precondition:** The user is signed in and is viewing the simulation dashboard.
- **Main flow:** The user selects a stock ticker and chooses to buy, sell, or place an order. The user inputs the quantity. The system validates the current simulated price against the user's available cash or stock holdings. The system executes the trade, updates the user's portfolio cash and holdings, and records a transaction log.
- **Alternative flow:** If the user lacks sufficient virtual cash for a buy, or sufficient shares for a sell, the system rejects the trade and shows an error toast.
- **Postcondition:** The user's portfolio and transaction history are updated to reflect the completed trade.

2.5.6 UC-06: View portfolio

- **Actor:** Registered User
- **Precondition:** The user is signed in.
- **Main flow:** The user navigates to the portfolio page. The system calculates and displays the current cash balance, the list of current stock holdings, the total portfolio value, realized and unrealized profit/loss, and a chart showing risk metrics and transaction history.

- **Alternative flow:** If the user has not made any trades yet, the portfolio simply shows the initial cash balance and an empty holdings list.
- **Postcondition:** The user has full visibility into their portfolio's financial health and performance.

2.5.7 UC-07: View leaderboard

- **Actor:** Registered User
- **Precondition:** The user is signed in.
- **Main flow:** The user opens the leaderboard page. The system aggregates portfolio values from all active users, sorts them by highest performance, and displays a ranked list. The user can see their own rank relative to others.
- **Alternative flow:** If the leaderboard data is temporarily unavailable, the system displays a loading state or a cached version of the rankings.
- **Postcondition:** The user's relative standing in the simulation is presented.

2.5.8 UC-08: Learn lesson

- **Actor:** Registered User
- **Precondition:** The user is signed in and navigates to the learning center.
- **Main flow:** The user browses available learning paths and selects a lesson. The user reads the educational material and marks sections as completed. The system tracks the user's learning progress and saves it to the database.
- **Alternative flow:** If the user loses network connection, progress tracking may fail to save immediately and will retry.
- **Postcondition:** The user's learning progress is updated and visually indicated in the learning dashboard.

2.5.9 UC-09: Take quiz

- **Actor:** Registered User
- **Precondition:** The user is signed in and has accessed a lesson containing a quiz.

- **Main flow:** The user answers the multiple-choice questions in the quiz and submits their answers. The system grades the submission, calculates the score, and saves the quiz result. The system displays immediate feedback showing correct and incorrect answers.
- **Alternative flow:** If the user submits the quiz with unanswered questions, the system may prompt them to complete all questions before grading.
- **Postcondition:** The user's quiz attempt and score are persistently stored.

2.5.10 UC-10: Join contest

- **Actor:** Contest Participant
- **Precondition:** The user is signed in and there is an active contest running.
- **Main flow:** The user views active contests and clicks join. The system initializes a separate, contest-specific portfolio with a predefined starting cash balance. The user navigates to the contest arena to trade specifically within the rules and allowed tickers of that contest.
- **Alternative flow:** If the contest has already ended or is full, the system prevents the user from joining and displays a relevant message.
- **Postcondition:** The user becomes a participant in the contest with an isolated contest portfolio.

2.5.11 UC-11: Ask chatbot

- **Actor:** Registered User
- **Precondition:** The user is signed in and the chatbot widget is enabled.
- **Main flow:** The user opens the chatbot panel and types a question regarding trading concepts or platform usage. The chatbot service processes the intent, incorporates current market/portfolio context, and returns an educational response along with relevant suggestion cards.
- **Alternative flow:** If the user asks for real-world financial advice, the chatbot detects the unsafe intent and responds with a strict disclaimer stating that it only provides simulated educational guidance.
- **Postcondition:** The chatbot's response is displayed to the user and the conversation history is saved.

2.5.12 UC-12: Manage market scenario

- **Actor:** Platform Maintainer
- **Precondition:** The maintainer has appropriate access to trigger administrative market actions.
- **Main flow:** The maintainer selects a predefined market scenario (e.g., tech bubble, market crash, or inflation) and activates it. The simulation engine adjusts price drift, volatility, and momentum parameters globally. The engine broadcasts the scenario change, affecting real-time prices for all users.
- **Alternative flow:** The maintainer decides to deactivate an ongoing scenario, returning the market to normal baseline behavior.
- **Postcondition:** The market simulation globally adopts the new pricing behavior dictated by the active scenario.

2.6 Functional Requirements

2.6.1 FR-01 Authentication and User Profile

- FR-AUTH-1 The system shall allow users to sign up using email, password, and display name.
- FR-AUTH-2 The system shall hash passwords before storage.
- FR-AUTH-3 The system shall allow users to sign in.
- FR-AUTH-4 The system shall allow users to view and update their display name.
- FR-AUTH-5 The system shall create a default portfolio for each new user.
- FR-AUTH-6 The system shall require users to verify their email address before signing in.

2.6.2 FR-02 Market Simulation

- FR-MKT-1 The system shall maintain a list of simulated stocks.
- FR-MKT-2 The system shall generate historical price bars for each stock.
- FR-MKT-3 The system shall stream real-time price updates through WebSocket.
- FR-MKT-4 The system shall provide stock quotes and historical data through REST APIs.

- FR-MKT-5 The system shall support market scenarios and regimes.

2.6.3 FR-03 Trading and Order Management

- FR-TRD-1 The system shall allow users to execute buy and sell trades.
- FR-TRD-2 The system shall reject buy trades when virtual cash is insufficient.
- FR-TRD-3 The system shall reject sell trades when holdings are insufficient.
- FR-TRD-4 The system shall record each successful transaction.
- FR-TRD-5 The system shall allow users to place, view, and cancel simulated orders.
- FR-TRD-6 The system shall allow users to open leveraged long or short positions.
- FR-TRD-7 The system shall allow users to close leveraged positions and calculate realized PnL.
- FR-TRD-8 The system shall liquidate open leveraged positions when losses exceed the liquidation threshold.

2.6.4 FR-04 Portfolio Management and Risk Analytics

- FR-PRT-1 The system shall display cash balance and current holdings.
- FR-PRT-2 The system shall calculate total portfolio value.
- FR-PRT-3 The system shall calculate realized and unrealized profit/loss.
- FR-PRT-4 The system shall display transaction history.
- FR-PRT-5 The system shall calculate risk metrics such as volatility, Sharpe ratio, and maximum drawdown.

2.6.5 FR-05 Learning Center

- FR-LRN-1 The system shall display learning paths and lessons.
- FR-LRN-2 The system shall track completed lesson sections.
- FR-LRN-3 The system shall allow users to submit quizzes.
- FR-LRN-4 The system shall save quiz results.
- FR-LRN-5 The system shall display badges, practice tasks, pattern game, and market analysis lab activities.

2.6.6 FR-06 Chatbot, Advisor, Blog, Contest, and Leaderboard

- FR-SOC-1 The system shall provide a chatbot widget for educational questions and platform guidance.
- FR-SOC-2 The system shall save and clear chatbot history.
- FR-SOC-3 The system shall display advisor-oriented educational responses and mock backtest output.
- FR-SOC-4 The system shall allow signed-in users to create, publish, archive, and delete their own blog posts.
- FR-SOC-5 The system shall allow users to vote and comment on published posts.
- FR-SOC-6 The system shall display active contests and allow users to join them.
- FR-SOC-7 The system shall maintain general and contest leaderboards.

2.7 Non-functional Requirements

Category	Requirement
Performance	The frontend should update displayed prices smoothly with the simulated tick interval.
Data integrity	Portfolio cash and holdings must not become negative after trades.
Reliability	If the WebSocket connection is unavailable, the frontend should continue displaying simulated movement or recover gracefully.
Security	Passwords must be hashed before storage.
Authorization	Blog editing, archiving, and deleting should be limited to the owner of the post.
Usability	Users should be able to navigate to Simulation, Portfolio, Learn, Blogs, Contest, and Leaderboard from the main navigation.
Maintainability	Simulation, order, portfolio, learning, chatbot, blog, contest, and leaderboard logic should be separated into routes, services, stores, and components.
Scalability	The system should allow adding more stocks, lessons, contests, and blog posts.
Portability	The system should run through Docker Compose or manual npm setup.
Ethical safety	Chatbot and advisor responses must clarify that the system is for educational simulation and not real financial advice.

Table 2.4: Non-functional Requirements

Chapter 3

Requirement Analysis

This chapter transforms the functional requirements into an analysis model that describes the main domain entities, system responsibilities, interactions, and persistent data structures of SoictStock. The analysis is based on the implemented React frontend, Express backend, WebSocket price stream, and MongoDB collections. It intentionally distinguishes implemented behavior from simulation-only or mock behavior to avoid overstating the current system scope.

3.1 Identifying Analysis Classes

The analysis classes are organized using the Boundary-Control-Entity pattern. Boundary classes represent user-facing interfaces, control classes coordinate use cases and business rules, and entity classes represent persistent or long-lived domain information.

Table 3.1: Main Boundary Classes

Boundary class	Responsibility
AuthModal	Collects sign-up and sign-in information, displays validation or authentication errors, and forwards valid credentials to the authentication store.
Navbar	Provides global navigation, exposes authentication actions, and displays the current user's account menu and entry points to the major system modules.
LandingPage	Introduces the platform, summarizes its educational and simulation features, and directs users to the simulator, learning center, portfolio, and community modules.

Continued on the next page.

Table 3.1 continued from the previous page.

Boundary class	Responsibility
SimulationPage	Presents the real-time trading terminal, including the watchlist, ticker information, candlestick chart, news panel, order form, order book, time-and-sales view, open positions, and pending orders.
PortfolioPage	Displays virtual cash, holdings, allocation, realized and unrealized profit/loss, performance history, transaction records, and downloadable portfolio information.
LearnPage	Presents learning paths, lessons, quizzes, practice activities, progress indicators, recommendations, and achievement badges.
LeaderboardPage	Displays ranked simulated portfolio performance and allows users to view available ranking periods.
BlogPages	Support browsing, filtering, reading, drafting, editing, publishing, archiving, voting on, and commenting on educational community posts.
ContestPages	Present active contests, reward information, contest enrollment, contest-specific trading, portfolio performance, countdown information, and contest leaderboards.
ChatbotWidget	Collects natural-language questions, presents assistant replies and suggestion cards, and provides conversation controls such as opening, moving, and clearing the chatbot panel.

Table 3.2: Main Control Classes

Control class	Responsibility
AuthenticationController	Validates registration and login requests, checks email uniqueness, hashes and verifies passwords, creates user records, initializes default portfolios, and returns safe profile data.
MarketController	Provides stock lists, current quotations, raw price history, and server-aggregated OHLCV candles for the requested ticker and timeframe.

Continued on the next page.

Table 3.2 continued from the previous page.

Control class	Responsibility
PriceStreamController	Manages WebSocket connections, reports initialization status, sends recent market history to newly connected clients, and broadcasts real-time price updates.
TradingController	Receives order requests, validates required order fields, delegates order placement or cancellation, and returns the current open and filled order sets.
PortfolioController	Validates virtual funds and owned shares, updates cash and holdings, records completed transactions, calculates current portfolio value, and refreshes leaderboard information.
LearningController	Creates, retrieves, updates, and resets learning progress; records completed lesson sections and quiz attempts for signed-in users.
BlogController	Coordinates draft creation, post editing, publication, archiving, deletion, filtering, voting, commenting, and public author-profile retrieval.
ContestController	Creates default contests when required, enrolls users, initializes contest portfolios, validates allowed tickers, executes contest trades, and computes contest rankings.
ChatbotController	Accepts chatbot messages, delegates response generation, stores recent conversation messages, retrieves saved history, and clears a user's conversation on request.
SimulationEngine	Initializes the stock universe, restores or generates five-minute market history, produces real-time simulated prices, applies active regimes and news effects, and notifies registered tick listeners.
OrderBookService	Executes market orders using simulated quotes and slippage, stores pending Limit and Stop-Loss orders, checks trigger conditions after price updates, and cancels eligible orders.
NewsInjector	Retrieves or generates market news, identifies affected sectors or tickers, assigns simulated sentiment and impact, stores news items, and applies bounded price shocks.

Continued on the next page.

Table 3.2 continued from the previous page.

Control class	Responsibility
RiskMetrics	Calculates available portfolio risk indicators, including volatility, Sharpe ratio, maximum drawdown, beta, win rate, and profit factor when sufficient observations are available.
ChatbotService	Produces a response through the configured language model when available, falls back to rule-based educational responses when necessary, and keeps answers within the simulation-only safety boundary.

Table 3.3: Main Entity Classes

Entity class	Responsibility
User	Represents a registered account, including the user's identifier, email, hashed password, display name, and account timestamps.
Portfolio	Represents a user's main virtual account, including available cash, initial cash, embedded holdings, and the latest update time.
Holding	Represents a position in one ticker, including owned shares, average acquisition price, and accumulated realized profit/loss.
Order	Represents a Market, Limit, or Stop-Loss instruction, including side, ticker, quantity, trigger or execution price, status, owner, and timestamps.
Transaction	Represents a completed buy or sell operation with its ticker, quantity, execution price, total value, order type, status, and creation time.
PortfolioSnapshot	Represents a historical observation of portfolio value used for performance charts and risk calculations.
LeaderboardEntry	Represents a user's current rank-related information, including portfolio value, total return, trade count, and available risk statistics.

Continued on the next page.

Table 3.3 continued from the previous page.

Entity class	Responsibility
Stock	Represents one fictional tradable company with its ticker, name, sector, base price, drift, volatility, and display attributes.
Quote	Represents the current simulated market view of a stock, including last price and derived bid, ask, spread, and change information.
Tick	Represents a stored five-minute market observation containing a ticker, Unix timestamp, simulated price, and simulated volume.
OHLCVBar	Represents an aggregated chart candle containing open, high, low, close, volume, and interval start time.
NewsItem	Represents a market news event with headline, source, sentiment, affected tickers, impact value, and publication timestamp.
MarketScenario	Represents the currently selected simulation regime and its drift or volatility overrides; it is runtime configuration rather than a dedicated MongoDB collection.
LearningProgress	Represents a user's complete learning state, including lesson progress, quiz results, earned badges, current level, and recommendation identifiers.
LessonProgress	Represents completed sections, completion state, completion time, and last-opened time for one lesson.
QuizResult	Represents quiz attempts, latest score, best score, pass status, and completion state for one quiz.
AchievementBadge	Represents a badge earned from learning or simulation activity together with its identifier and award time.
BlogPost	Represents an educational community post, including title, content, excerpt, author, publication status, stock tags, rating counters, and timestamps.
Comment	Represents an embedded response to a published blog post, including author information, content, and creation time.

Continued on the next page.

Table 3.3 continued from the previous page.

Entity class	Responsibility
Vote	Represents a user’s current positive, negative, or neutral vote on a published blog post.
PublicProfile	Represents the publicly visible author information and the set of published posts associated with that author.
Contest	Represents an active competition, including its name, status, end time, allowed tickers, custom stocks, and displayed rewards.
ContestPortfolio	Represents a participant’s contest-specific virtual cash, holdings, initial cash, current value, and enrollment time.
ContestTransaction	Represents a completed contest buy or sell operation associated with a contest and participant.
ContestReward	Represents the rank range, reward type, name, image, and description displayed for a contest prize.
ChatbotConversation	Represents one persisted conversation per user, including recent messages and creation or update timestamps.
ChatMessage	Represents a user or assistant message with its role, content, intent, suggestions, cards, metadata, and creation time.
SuggestionCard	Represents an optional lesson, quiz, portfolio insight, consultant, or navigation card attached to a chatbot response.

3.1.1 Responsibilities of Key Control Classes

- **SimulationEngine** initializes the stock universe, restores or generates five-minute historical bars, updates in-memory prices every three seconds, stores one bar per five-minute bucket, applies scenario parameters and bounded news shocks, and notifies WebSocket listeners.
- **OrderBookService** accepts Market, Limit, and Stop-Loss orders. Market orders are filled using a simulated bid/ask quote and slippage, whereas pending orders are stored and checked against each subsequent price update.
- **PortfolioController** validates virtual funds or shares, updates holdings and average cost, records transactions, and updates the general leaderboard after a completed trade.

- **LearningController** stores lesson and quiz progress while the frontend determines recommendations, levels, and badge eligibility from application rules.
- **ChatbotController** builds a context-aware request, delegates response generation to the backend chatbot service, applies simulation-safe language, and persists recent conversation history.

3.1.2 BCE Analysis Diagram

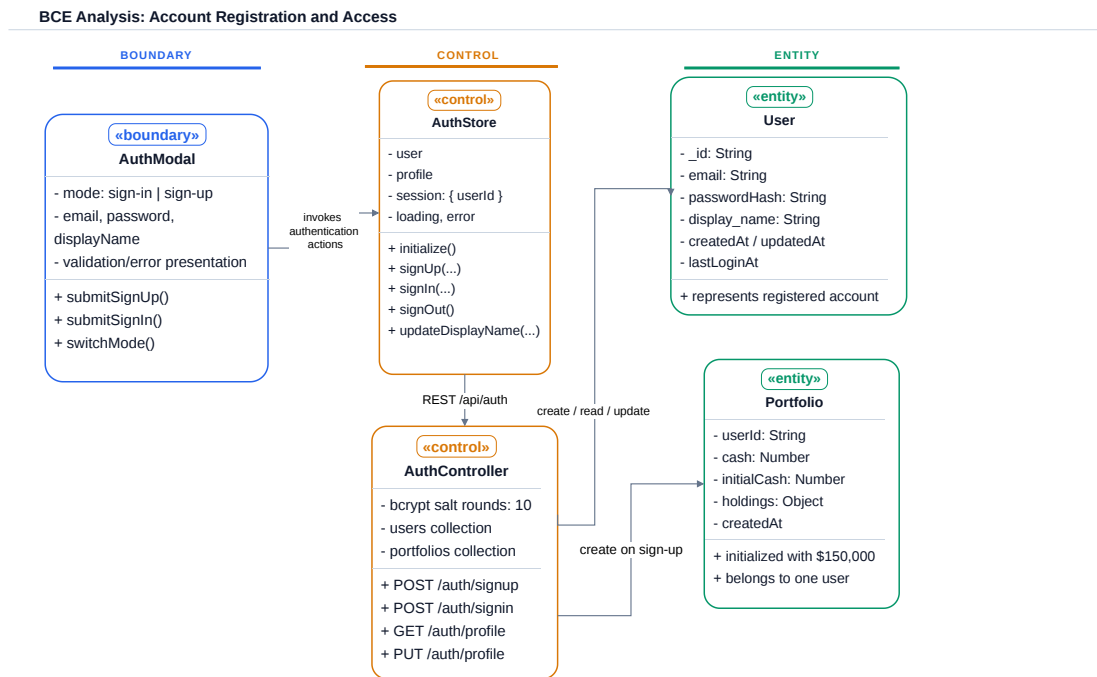


Figure 3.1: Boundary–Control–Entity analysis diagram for SoictStock

BCE Analysis: Market Simulation and Real-Time Streaming

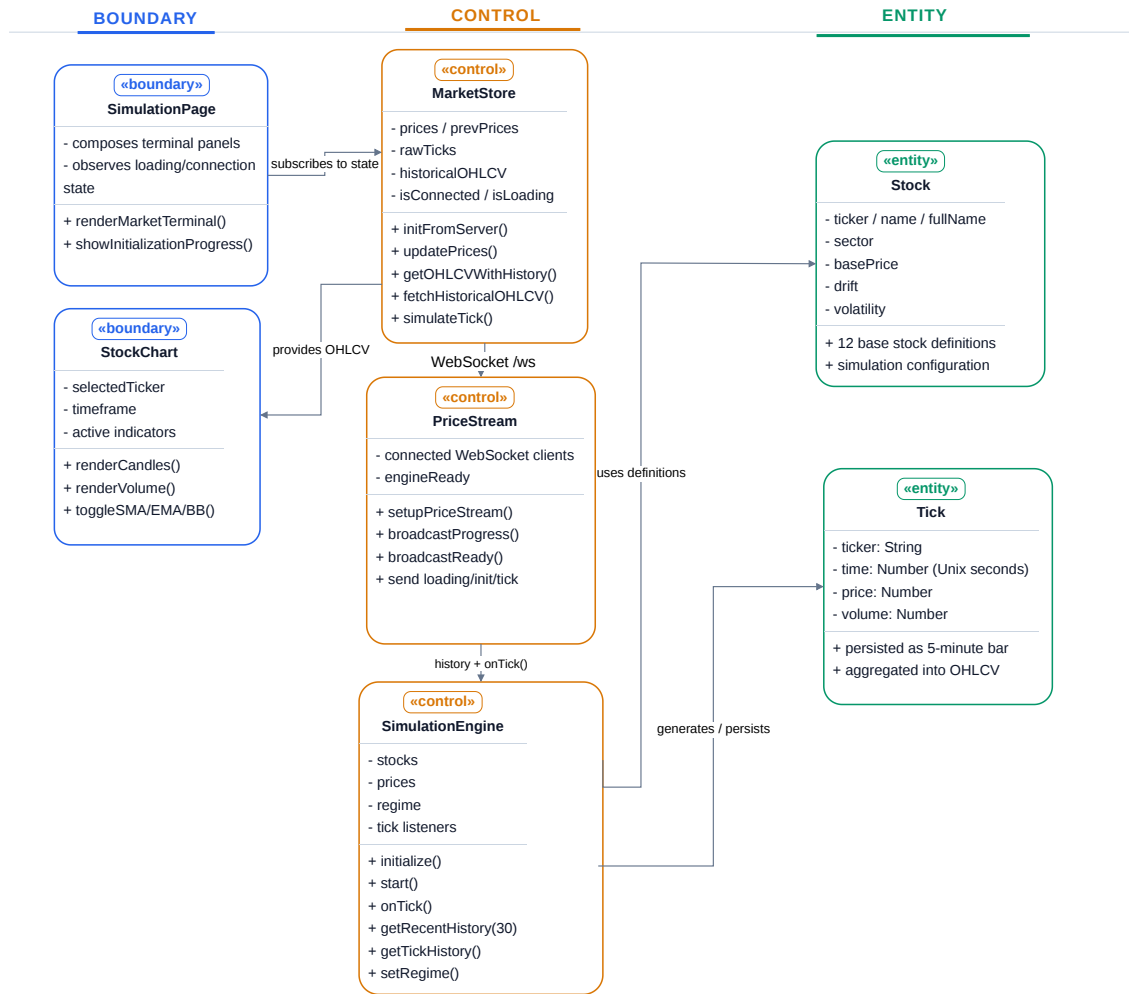
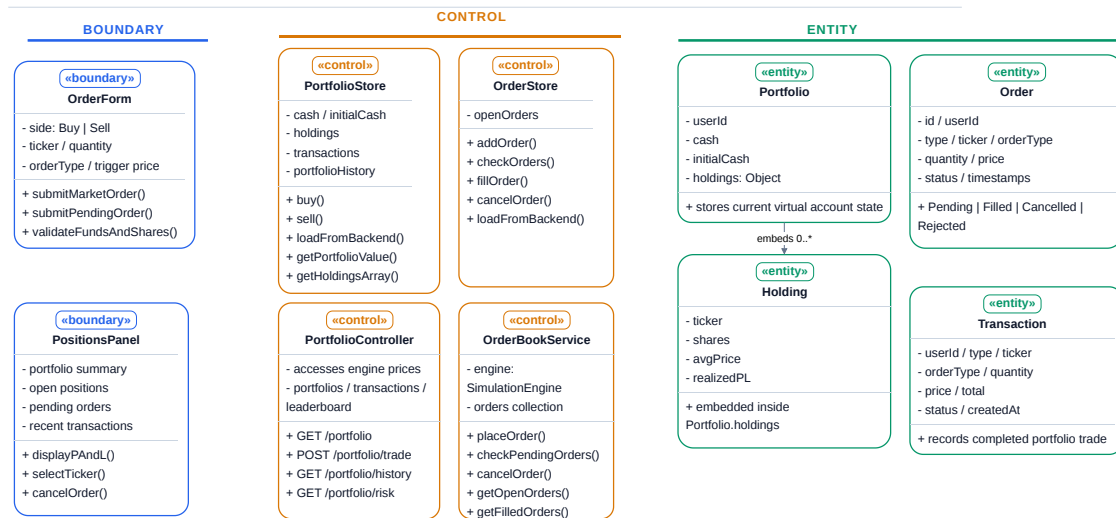


Figure 3.2: Boundary–Control–Entity analysis diagram for SoictStock

BCE Analysis: Trading, Pending Orders, and Portfolio



Runtime Interaction Map

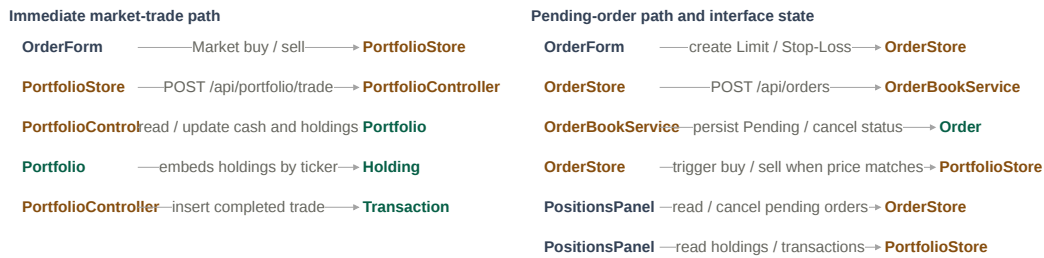


Figure 3.3: Boundary–Control–Entity analysis diagram for SoictStock

BCE Analysis: Learning Content and Progress Tracking

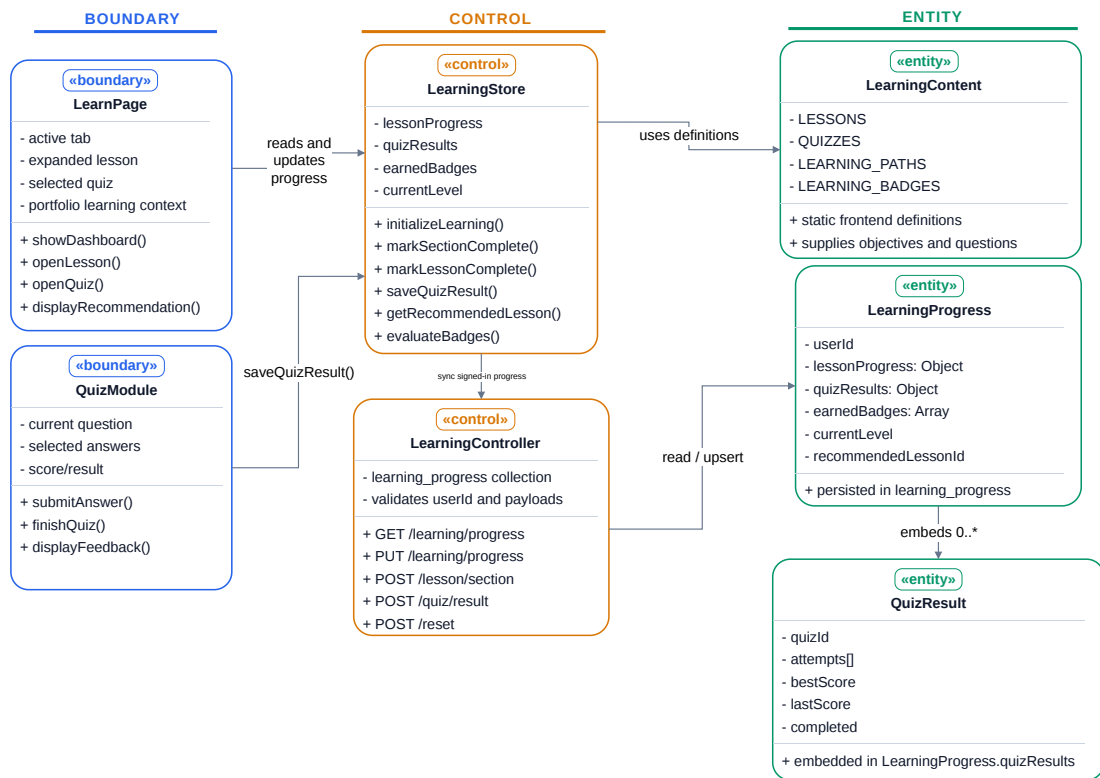


Figure 3.4: Boundary–Control–Entity analysis diagram for SoictStock

BCE Analysis: AI Learning Assistant and Conversation History

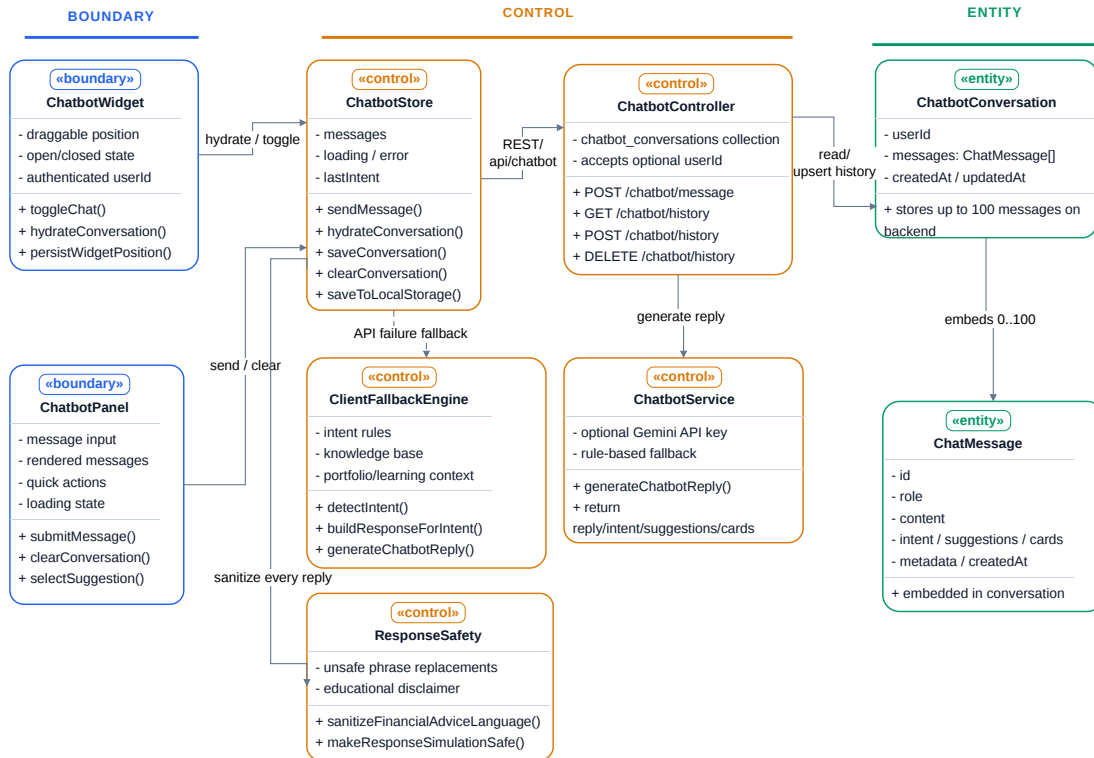


Figure 3.5: Boundary–Control–Entity analysis diagram for SoictStock

3.2 Main Analysis Scenarios

The following scenarios capture the major interactions that the analysis model must support.

Scenario 1: Register an account. A guest submits an email, password, and display name. The system validates uniqueness, hashes the password, creates the user, and initializes a portfolio with \$150,000 virtual cash.

Scenario 2: Open the simulation terminal. The frontend connects to the WebSocket endpoint, receives initial price history and current prices, then applies subsequent three-second tick updates to the market store and chart.

Scenario 3: Execute a market trade. A user selects a stock, side, and quantity. The system validates virtual funds or owned shares, updates the portfolio, records the transaction, and refreshes portfolio and leaderboard values.

Scenario 4: Manage a pending order. A user places a Limit or Stop-Loss order. The order remains pending until its trigger condition is met

or the user cancels it.

Scenario 5: Review portfolio performance. A user views cash, holdings, average cost, current market value, realized and unrealized profit/loss, allocation, transaction history, and available risk metrics.

Scenario 6: Observe market events. The news service fetches or generates an event, identifies affected stocks, applies a bounded simulated price shock, stores the event, and displays it in the news panel.

Scenario 7: Activate a market scenario. A user selects a named scenario. The system changes the active simulation regime and its drift or volatility parameters until the scenario is deactivated.

Scenario 8: Complete learning activities. A learner opens a lesson, completes sections, submits a quiz, and receives updated progress, recommendation, level, and badge information.

Scenario 9: Use the learning chatbot. A user asks about financial concepts, portfolio risk, order types, lessons, or platform navigation. The system returns simulation-safe guidance and optionally persists the conversation.

Scenario 10: Publish community content. A signed-in user creates a draft, edits and publishes it, while other users may filter posts, vote, comment, and view the author's public profile.

Scenario 11: Join a contest. A signed-in user joins an active contest, receives a separate \$100,000 contest portfolio, trades only allowed tickers, and appears in the contest leaderboard.

3.3 Sequence Diagrams

The sequence diagrams below focus on the workflows that contain the most important coordination logic. UI-only interactions such as filtering a watchlist are omitted because they do not involve multiple analysis objects.

3.3.1 User Registration

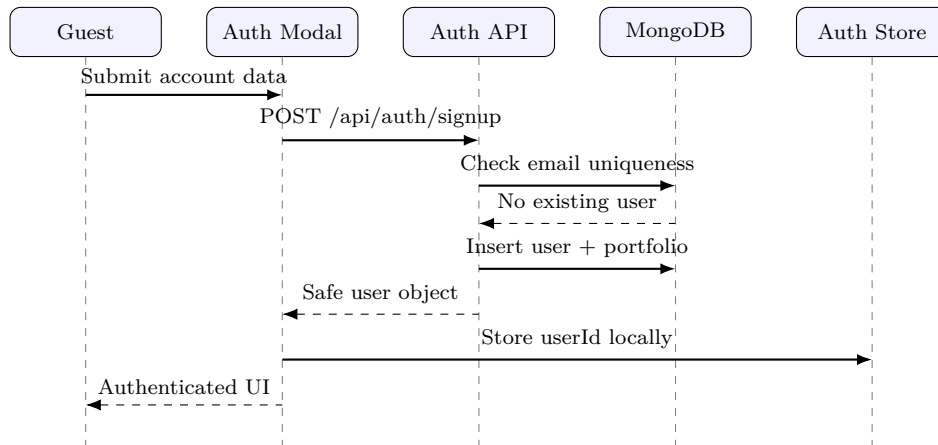


Figure 3.6: Sequence diagram for user registration

3.3.2 Real-Time Market Streaming

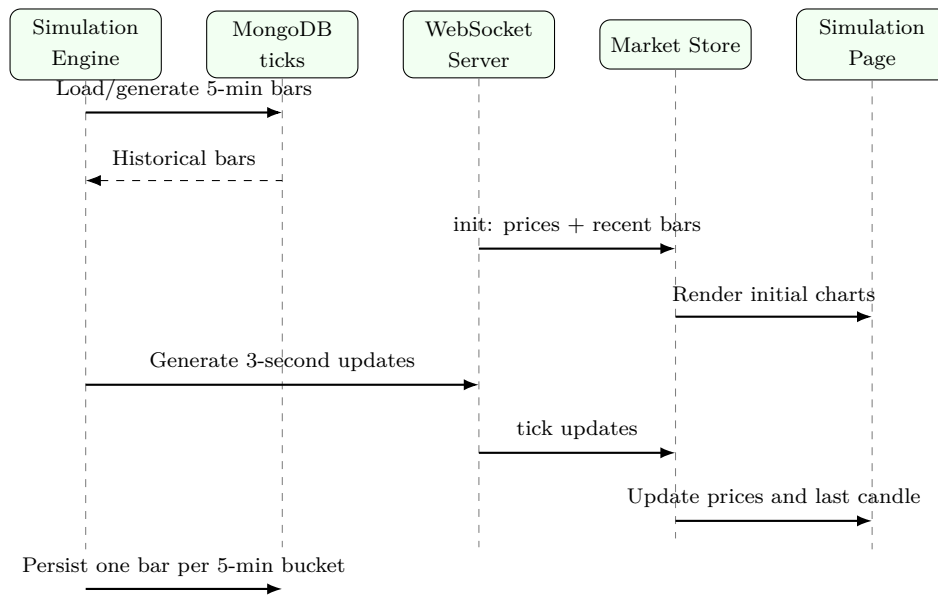


Figure 3.7: Sequence diagram for real-time market streaming

3.3.3 Market Trade Execution

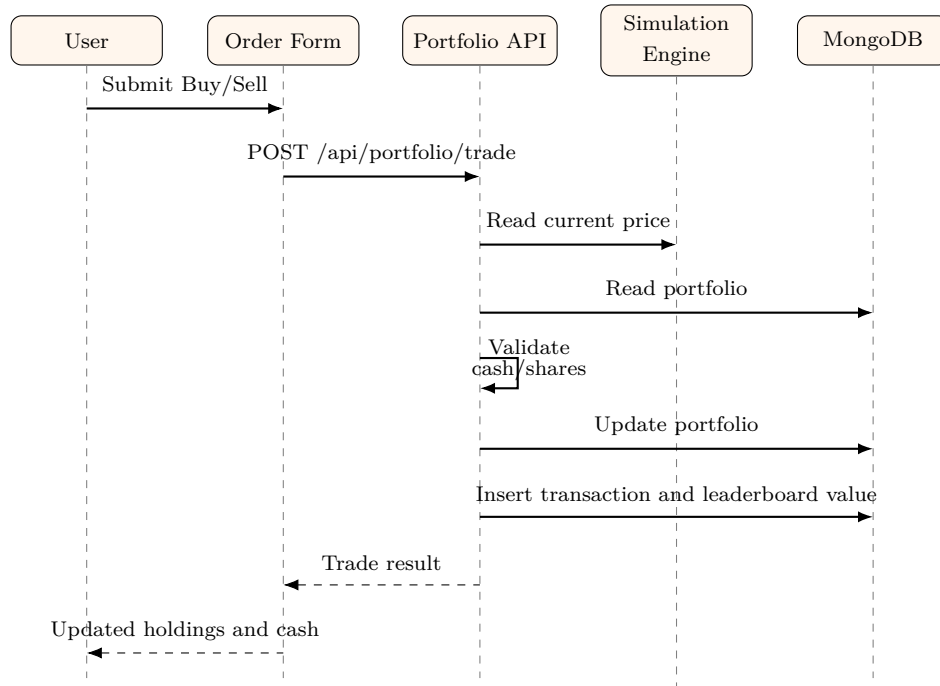


Figure 3.8: Sequence diagram for executing a market trade

3.3.4 Pending Order Lifecycle

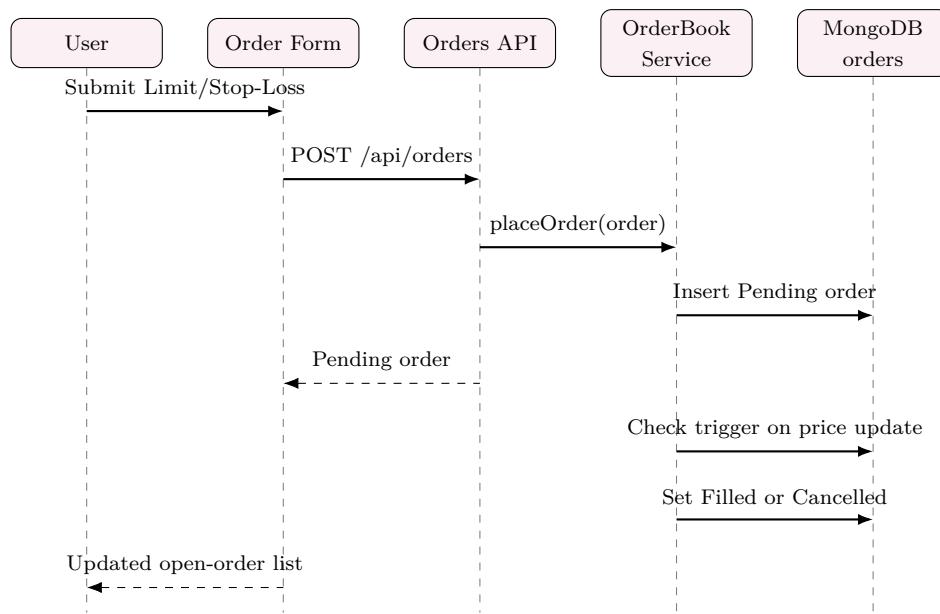


Figure 3.9: Sequence diagram for a pending order

3.4 Logical Data Model

Although MongoDB is document-oriented rather than relational, an ERD-style logical model is still useful for showing ownership and reference relationships. Embedded structures are explicitly marked to avoid presenting them as separate collections.

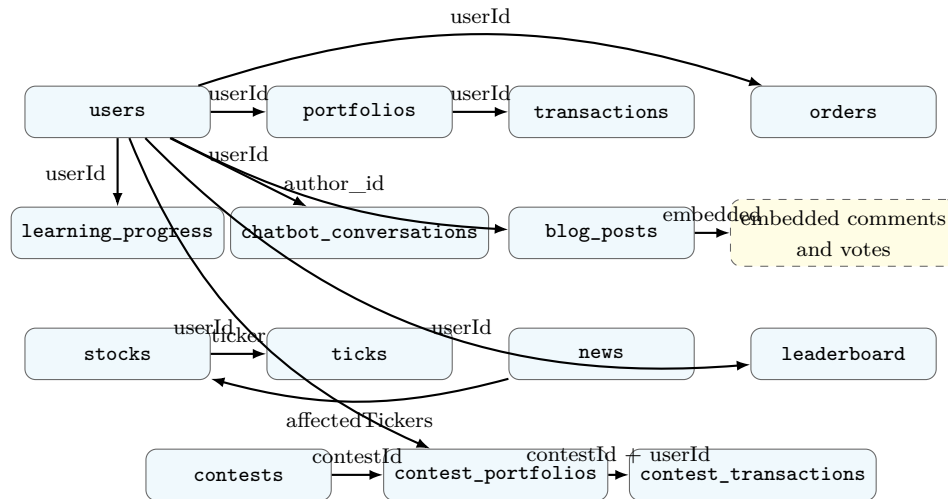


Figure 3.10: ERD-style logical data model for MongoDB collections

The logical model has several document-oriented characteristics:

- `portfolios.holdings` is an object keyed by ticker and embeds shares, average price, and realized profit/loss.
- `blog_posts` embeds comments and a per-user vote map.
- `learning_progress` embeds lesson progress, quiz attempts, badges, and recommendations.
- `chatbot_conversations` embeds recent messages for one user.
- Contest rankings are derived from `contest_portfolios`; there is no separate contest leaderboard collection.
- Market scenarios are runtime configuration of the simulation engine; there is no persistent `scenarios` collection in the current implementation.

3.5 Data Dictionary

Tables 3.4–3.6 summarize the fields that are most relevant to the analysis model. MongoDB may also generate an internal `_id` field where the application does not assign one explicitly.

Table 3.4: Core user, portfolio, and trading data

Collection	Field	Type	Description
users	_id	String	Application-generated user identifier.
	email	String	Lowercase login email; uniquely indexed.
	passwordHash	String	bcrypt password hash.
	display_name	String	Public display name.
	createdAt,	ISO Date-	Account audit timestamps.
	updatedAt,	Time String	
	lastLoginAt		
portfolios	userId	String	Owner; uniquely indexed.
	cash	Number	Remaining virtual cash.
	initialCash	Number	Initial portfolio value, normally 150000.
	holdings	Object	Ticker-keyed embedded holdings.
	holdings.<ticker>.shares	Number	Number of owned shares.
	holdings.<ticker>.avgPrice	Number	Weighted average purchase price.
	holdings.<ticker>.realizedPL	Number	Accumulated realized profit/loss.
transactions	createdAt,	ISO Date-	Creation and last modification time.
	updatedAt	Time String	
	userId	String	Transaction owner.
	type	Enum	Buy or Sell.
	ticker	String	Traded stock symbol.
	orderType	String	Order type reported for the completed trade.
	quantity, price, total	Number	Executed quantity, price, and total value.
	status	String	Usually Filled.
	createdAt	ISO Date-	Execution time.
		Time String	
orders	_id, id	String	Application-generated order identifier.
	userId	String	Order owner.

Collection	Field	Type	Description
	type	Enum	Buy or Sell.
	orderType	Enum	Implemented backend values: Market, Limit, Stop-Loss.
	quantity, price	Number	Requested quantity and optional trigger/limit price.
	status	Enum	Pending, Filled, Rejected, or Cancelled.
	createdAt, filledAt	ISO Date-Time String	Creation and optional fill time.
	executionPrice, slippage	Number	Fill details when available.

Table 3.5: Market, analytics, and content data

Collection	Field	Type	Description
stocks	_id, ticker	String	Ticker is used as the document identifier.
	name, fullName, sector	String	Display and classification fields.
	basePrice, drift, volatility	Number	Simulation parameters.
ticks	ticker	String	Stock reference.
	time	Integer	Unix timestamp in seconds for a five-minute bar.
	price, volume	Number	Simulated closing price and volume.
news	_id, id	String	Application-generated news identifier.
	headline, description, source	String	Displayed article information.
	sentiment	Enum	Positive, negative, or neutral.
	affectedTickers	Array<String>	Stocks affected by the simulated event.
	isMarketWide	Boolean	Whether the event affects the whole market.

Collection	Field	Type	Description
	impact	Number	Simulated shock magnitude before engine-side capping.
	timestamp	ISO Date-Time String	Event creation time.
leaderboard	userId	String	Ranked user.
	portfolioValue, totalReturn	Number	Current value and return percentage.
	trades, sharpe	Number	Trade count and optional Sharpe ratio.
	updatedAt	ISO Date-Time String	Last ranking update.

Table 3.6: Learning, chatbot, blog, and contest data

Collection	Field	Type	Description
learning_progress	userId	String	Learner identifier; uniquely indexed.
	lessonProgress	Object	Per-lesson completed sections and completion timestamps.
	quizResults	Object	Per-quiz attempts, best score, latest score, and completion state.
	earnedBadges	Array	Badge identifiers and optional earning time.
	currentLevel	String	Current calculated learning level.
	recommendedLessonId, recommendedChallengeId	String or Null	Stored recommendation identifiers.
chatbot_conversations	userId	String	Conversation owner; uniquely indexed.
	messages	Array<Object>	Up to 100 persisted user and assistant messages.
	messages.role, messages.content	String	Speaker role and message text.
	messages.intent, messages.cards	String/Array	Optional detected intent and UI cards.

Collection	Field	Type	Description
	createdAt, updatedAt	ISO Date- Time String	Conversation timestamps.
blog_posts	_id	ObjectId	MongoDB-generated post identifier.
	author_id	String	Reference to <code>users._id</code> .
	title, slug, excerpt, content	String	Post content and unique public slug.
	status	Enum	Draft, published, or archived.
	stock_tags	Array<String>	Normalized ticker tags.
	upvotes, downvotes, rating	Number	Voting counters.
	votes_by_user	Object	Map from user ID to current vote.
	comments	Array<Object>	Embedded comments.
	created_at, updated_at, published_at	ISO Date- Time String or Null	Post lifecycle timestamps.
contests	_id	ObjectId	Contest identifier.
	name, status, endTime	String/DateTime	Contest identity and availability.
	allowedTickers	Array<String>	Tradable tickers for the contest.
	customStocks, rewards	Array<Object>	Contest-specific stocks and displayed rewards.
contest_portfolios	contestId, userId	String	Composite unique participation key.
	cash, initialCash, portfolioValue	Number	Contest cash and valuation fields.
	holdings	Object	Ticker-keyed contest holdings.
contest_transactions	contestId, userId	String	Contest and participant references.
	type, ticker, quantity, price, total	Mixed	Executed contest trade details.

Collection	Field	Type	Description
	createdAt	ISO Date-Time String	Trade time.

3.6 Implementation Scope and Consistency Notes

The following points should be reflected consistently in later design, implementation, and testing chapters.

- The base market contains **12 fictional stocks across six sectors**. Active contests may inject additional custom stocks at runtime.
- The backend stores approximately **30 days of five-minute bars** in the current engine configuration. Three-second price updates are kept in memory and broadcast through WebSocket rather than persisted individually.
- The pricing model is a **Merton jump-diffusion simulation** with sector-specific jump parameters, soft price bounds, optional regime overrides, and bounded news shocks.
- The backend order service implements **Market, Limit, and Stop-Loss** orders. Stop-Limit and Take-Profit appear in the current frontend order form but are not fully executed by the backend and should therefore be treated as planned extensions unless implemented before submission.
- The general portfolio store currently uses the default backend user identifier in some synchronization calls. User-specific authentication and portfolio ownership should be verified during integration testing.
- Authentication persists a user identifier in browser local storage and validates it through the profile endpoint. The current system does not implement JWT, server-side sessions, or signed cookies.
- The advisor endpoint and backtest return predefined or synthetic educational results. The chatbot is separate: it may use a configured Gemini model and otherwise falls back to rule-based responses.
- The displayed order-book depth and Time & Sales rows are simulated UI visualizations derived from the current price rather than a true exchange matching engine.

Chapter 4

System Design

This chapter presents the detailed design of SoictStock based on the implemented source code. The system is designed as a modular web application that separates the React single-page application, the Express backend API, the real-time market simulation engine, and the MongoDB persistence layer. Compared with the analysis model in Chapter 3, this chapter focuses on implementation-level design decisions: runtime architecture, component responsibility, package organization, API boundary, database collections, deployment topology, and important consistency notes.

The design is intentionally simulation-oriented. All stocks, cash balances, prices, market events, orders, contests, rewards, advisor responses, and chatbot outputs are for educational use only. The system does not connect to a broker, does not execute real trades, and does not provide legally binding financial advice.

4.1 Selected Architecture

SoictStock uses a client-server architecture with real-time streaming. The client side is a React/Vite single-page application. It is responsible for navigation, page rendering, local UI state, form handling, chart rendering, fallback price animation, and interaction with the backend. The backend is an Express.js application that exposes REST endpoints and a WebSocket endpoint. It connects to MongoDB through a singleton database service and stores long-lived data such as users, portfolios, orders, transactions, contests, blog posts, learning progress, chatbot conversations, and five-minute simulated market bars.

The backend starts by loading environment variables, connecting to MongoDB, creating the Express application and HTTP server, attaching a WebSocket server at `/ws`, constructing shared services, mounting route modules, initializing the simulation engine, broadcasting market readiness to connected clients, starting the three-second real-time tick loop, starting the news injector, and scheduling liquidation checks for leveraged positions. The major backend services are created once and placed in `app.locals`;

route modules then use these shared service instances rather than constructing independent engines.

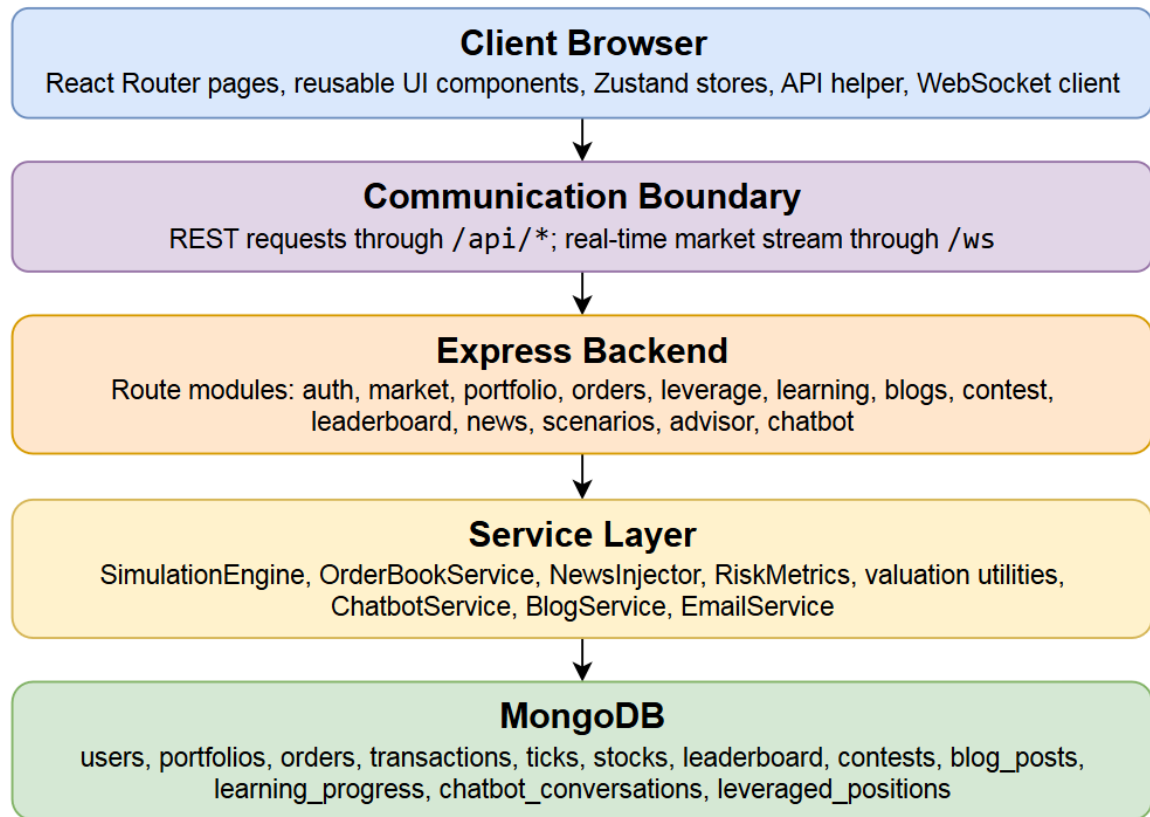


Figure 4.1: Implementation-level architecture of SoictStock

4.2 Main Components

Table 4.1 maps the major design components to their implementation files. The mapping is useful because it connects the system design chapter directly to the source code structure.

Table 4.1: Main design components and implementation mapping

Component	Design responsibility	Representative implementation files
Authentication and profile	Register accounts, hash passwords, verify email tokens, sign in users, update display names, and initialize the default virtual portfolio with \$150,000.	backend/routes/auth.js backend/services/emailService.js frontend/src/store/authStore.js frontend/src/components/shared/AuthModal.jsx frontend/src/pages/VerifyEmail.jsx
Market simulation and streaming	Seed the fictional stock universe, restore or generate five-minute historical bars, maintain current prices, apply regimes/scenarios/news shocks, and broadcast real-time ticks.	backend/services/simulationEngine.js backend/services/stockData.js backend/routes/market.js backend/routes/scenarios.js backend/websocket/priceStream.js frontend/src/store/marketStore.js
Trading and order management	Execute direct buy/sell trades, persist market/limit/stop-loss orders, check pending order triggers, update cash and holdings, and record filled transactions.	backend/routes/portfolio.js backend/routes/orders.js backend/services/orderBook.js frontend/src/store/orderStore.js frontend/src/store/portfolioStore.js frontend/src/components/simulation/OrderForm.jsx
Leverage positions	Open and close simulated long/short leveraged positions, reserve margin, calculate liquidation prices, and periodically liquidate positions when losses exceed the configured threshold.	backend/routes/leverage.js frontend/src/store/leverageStore.js frontend/src/components/simulation/PositionsPanel.jsx

Component	Design responsibility	Representative implementation files
Portfolio and risk analytics	Compute total portfolio value from cash, holdings, current prices, and open leveraged positions; retrieve portfolio history; calculate volatility, Sharpe ratio, maximum drawdown, beta, win rate, and profit factor when data is sufficient.	backend/routes/portfolio.js backend/services/riskMetrics.js backend/services/valuation.js frontend/src/pages/Portfolio.jsx frontend/src/store/portfolioStore.js
Leaderboard	Rank users by portfolio value and return, enrich entries with display names and current values, and update ranking after trades or position changes.	backend/routes/leaderboard.js backend/services/valuation.js frontend/src/pages/Leaderboard.jsx frontend/src/store/leaderboardStore.js
Learning center	Store lesson and quiz progress, save completed lesson sections, save quiz results, reset progress, and render lessons, dashboards, badges, practice tasks, pattern games, and market analysis activities.	backend/routes/learning.js frontend/src/lib/learningData.js frontend/src/store/learningStore.js frontend/src/pages/Learn.jsx frontend/src/components/learn/*.jsx
Chatbot and advisor	Process educational chatbot messages, store and clear conversation history, generate suggestion cards, provide mock advisor chat, and return synthetic backtest results.	backend/routes/chatbot.js backend/services/chatbotService.js backend/routes/advisor.js frontend/src/components/chatbot/*.jsx frontend/src/lib/chatbot*.js

Component	Design responsibility	Representative implementation files
Blog and community	Create, edit, publish, archive, delete, vote, comment on, filter, and browse educational blog posts and public author profiles.	<code>backend/routes/blogs.js</code> <code>backend/services/blogService.js</code> <code>frontend/src/pages/Blogs.jsx</code> <code>frontend/src/pages/BlogDetail.jsx</code> <code>frontend/src/pages/BlogEditor.jsx</code> <code>frontend/src/pages/UserProfile.jsx</code>
Contest module	Create or retrieve active default contests, enroll users, initialize contest portfolios, add custom stocks, execute contest trades, and compute contest-specific leaderboards.	<code>backend/routes/contest.js</code> <code>frontend/src/pages/Contest.jsx</code> <code>frontend/src/pages/ContestArena.jsx</code> <code>frontend/src/store/contestStore.js</code>
News and scenario module	Retrieve stored or generated news, apply news impact to simulation prices, activate/deactivate market scenarios, and show the market context in the trading terminal.	<code>backend/routes/news.js</code> <code>backend/services/newsInjector.js</code> <code>backend/services/newsService.js</code> <code>frontend/src/store/newsStore.js</code> <code>frontend/src/components/simulation/NewsPanel.jsx</code> <code>frontend/src/components/simulation/ScenarioSelector.jsx</code>

4.3 Component Diagram

The component diagram in Figure 4.2 illustrates the main runtime structure of Soict-Stock. The system is divided into three boundaries: the React frontend, the Express backend, and MongoDB data storage. Frontend components communicate with the backend only through the REST API and WebSocket price stream, while MongoDB is accessed exclusively by backend components. This separation keeps UI logic, business logic, and persistent data clearly isolated.

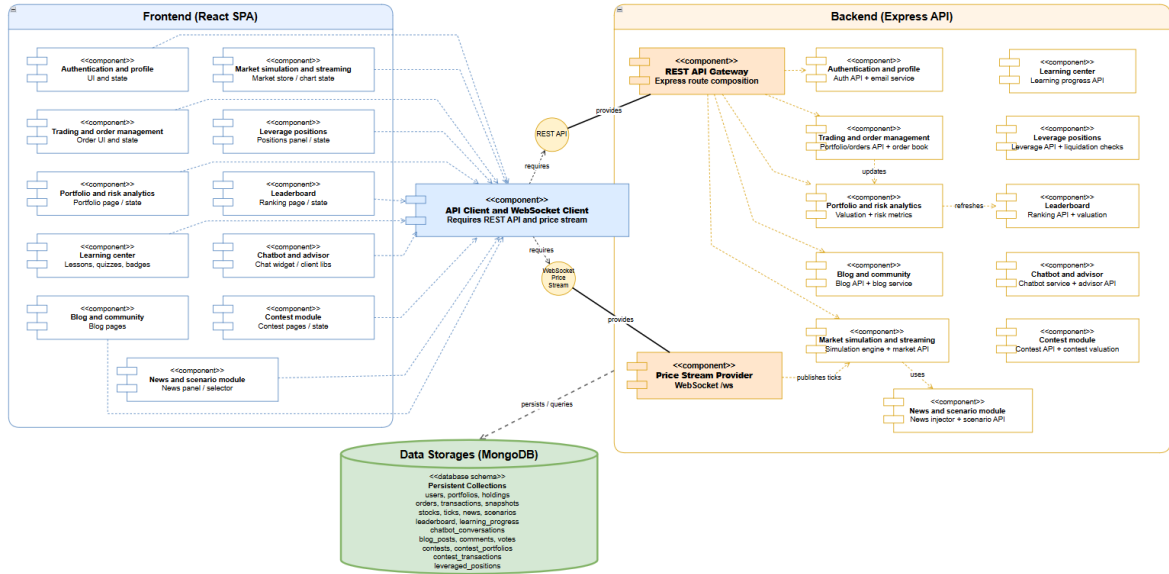


Figure 4.2: Runtime component diagram of SoictStock

4.4 Database Design

SoictStock uses a document-oriented MongoDB database. This design is suitable because several core objects are naturally nested. For example, a portfolio contains a map of holdings by ticker, a blog post contains embedded comments and vote counters, a learning progress document contains lesson and quiz progress, and a chatbot conversation contains recent messages.

4.4.1 Main Collections

Table 4.2: MongoDB collections used by the implementation

Collection	Purpose	Important fields / notes
users	Stores registered accounts and profile information.	_id, email, passwordHash, display_name, emailVerified, verificationToken, timestamps.
portfolios	Stores the main virtual portfolio for each user.	userId, cash, initialCash, holdings. The userId index is unique.
orders	Stores pending, filled, and cancelled order records.	id, userId, ticker, type, orderType, quantity, price, status.

Collection	Purpose	Important fields / notes
transactions	Stores completed direct buy/sell trades.	userId, type, ticker, quantity, price, total, status, createdAt.
ticks	Stores simulated five-minute market bars.	ticker, time, price, volume/OHLC-related fields. Indexed by ticker and time.
stocks	Stores the seeded fictional stock universe.	Twelve default fictional stocks across Technology, Healthcare, Energy, Finance, Consumer, and Industrial sectors.
leaderboard	Stores general platform rankings.	userId, portfolioValue, total return, trade count, risk statistics.
news	Stores generated or retrieved news events used by the news panel and news impact logic.	headline, sentiment, affectedTickers, impact, timestamp.
blog_posts	Stores draft, published, and archived community posts.	content, excerpt, cover_image_url, slug, status, author_id, stock_tags, comments, votes, rating counters.
learning_progress	Stores one learning-progress document per user.	userId, completed sections, quiz results, badges, level, recommendations.
chatbot_conversations	Stores recent chatbot conversation history.	userId, messages.
contests	Stores active and historical trading contests.	name, status, endTime, allowed tickers, custom stocks, rewards, scenario.
contest_portfolios	Stores contest-specific virtual portfolios.	Unique by contestId and userId; contains cash, holdings, initial cash, and portfolio value.

Collection	Purpose	Important fields / notes
contest_transactions	Stores trades executed inside contest arenas.	contestId, userId, ticker, trade type, quantity, price, total.
leveraged_positions	Stores open, closed, or liquidated leveraged long/short positions.	userId, contestId, ticker, side, leverage, margin, liquidationPrice, status.

4.4.2 Collection Relationship Diagram

The database does not enforce foreign keys as in a relational database. However, the code uses logical references such as `userId`, `contestId`, `ticker`, and `slug`. Figure 4.3 summarizes these logical relationships.

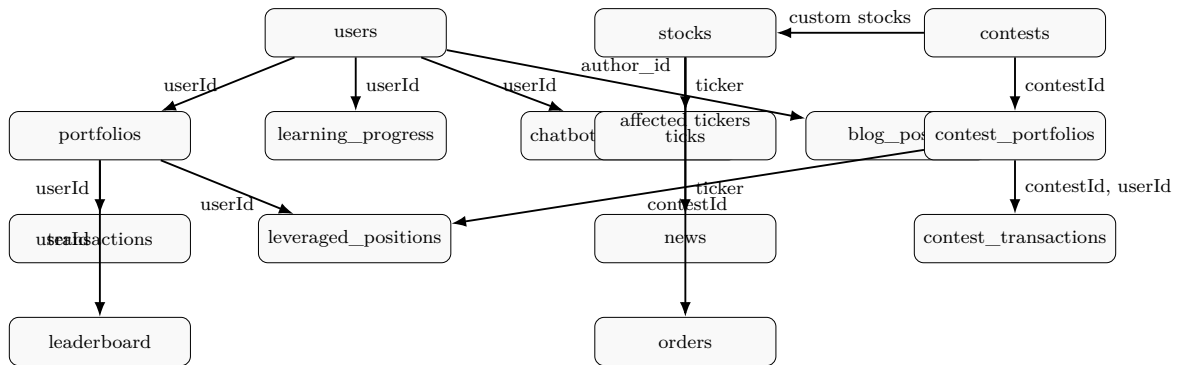


Figure 4.3: Logical MongoDB collection relationship diagram

4.4.3 Index Design

The database service creates indexes during startup. The important indexes are designed around frequent access patterns:

- `ticks`: indexed by `ticker`, `time` and `ticker`, `time desc` for historical chart retrieval and latest-price restoration.
- `orders`: indexed by `userId`, `status` for retrieving pending and filled user orders.
- `transactions`: indexed by `userId`, `createdAt desc` for portfolio transaction history.
- `portfolios`: unique index on `userId` to ensure one main portfolio per user.
- `portfolio_snapshots`: indexed by `userId`, `createdAt desc` for portfolio history and risk calculations.

- **leaderboard**: indexed by `portfolioValue desc` for ranking queries.
- **users**: unique index on `email`, plus indexes on `display_name` and sparse `verificationToken`.
- **blog_posts**: unique index on `slug`, plus indexes on `status`, `author`, `published time`, `rating`, `upvotes`, and `stock tags`.
- **learning_progress** and **chatbot_conversations**: unique index on `userId`.
- **contest_portfolios**: unique compound index on `contestId`, `userId` and ranking index on `contestId`, `portfolioValue desc`.
- **leveraged_positions**: indexes on `userId`, `status` and `contestId`, `userId`.

4.5 Detailed Package Design

4.5.1 Backend Package Design

The backend package follows a route-service structure. Route files validate HTTP requests, call shared services, and format responses. Service files contain reusable logic such as database connection, simulation, order execution, news injection, risk metrics, chatbot generation, blog operations, and portfolio valuation.

```

backend/
  server.js: application bootstrap, service construction, route mount-
ing, WebSocket setup
  routes/: REST controllers for auth, market, portfolio, orders, lever-
age, learning, blogs, contest, news, scenarios, advisor, chatbot, leader-
board
  services/: database singleton, simulation engine, order book, risk
metrics, valuation, blog service, chatbot service, news injector, email ser-
vice, stock data
  websocket/: WebSocket price stream setup
  Dockerfile, package.json, railway.toml: runtime and deployment
configuration

```

Figure 4.4: Backend package organization

Important backend design decisions are as follows:

- **Shared service instances.** `SimulationEngine`, `OrderBookService`, `NewsInjector`, and `RiskMetrics` are constructed in `server.js` and stored in `app.locals`. This avoids duplicate simulation states across routes.

- **Database singleton.** `services/db.js` manages the MongoDB client and database instance. Route and service modules call `getDb()` after startup connection is established.
- **Route-level modularity.** Each route file corresponds to a business module. This supports maintainability because authentication, market, portfolio, blog, learning, contest, and chatbot changes are localized.
- **Simulation persistence strategy.** MongoDB stores five-minute bars. The three-second real-time ticks are kept in memory and broadcast to clients. This reduces storage pressure while still supporting live visualization.
- **Periodic jobs.** The server starts the real-time simulation loop, the news injector, and a 30-second leveraged-position liquidation check.

4.5.2 Frontend Package Design

The frontend package follows a page-component-store structure. Pages represent major screens, components represent reusable UI blocks, stores hold client-side state and actions, and `lib/api.js` centralizes backend calls.

frontend/src/

App.jsx: route configuration, global WebSocket connection, periodic side effects

pages/: Landing, Simulation, Portfolio, Learn, Blogs, BlogDetail, BlogEditor, Contest, ContestArena, Leaderboard, VerifyEmail, UserProfile

components/: layout, shared controls, simulation terminal widgets, learning widgets, chatbot widgets

store/: Zustand stores for auth, market, portfolio, order, leverage, learning, news, contest, leaderboard, chatbot, settings

lib/: API helper, constants, formatters, indicators, learning data, chatbot engine/knowledge/safety/context utilities

main.jsx, index.css: React entry point and global style

Figure 4.5: Frontend package organization

Important frontend design decisions are as follows:

- **Single-page navigation.** `App.jsx` maps URLs to pages and keeps common layout elements such as `Navbar`, `Footer`, `ChatbotWidget`, and `Toast` outside page-specific routes.
- **Centralized API helper.** All backend calls are wrapped in `frontend/src/lib/api.js`, making route changes easier to maintain.

- **Module stores.** Zustand stores keep market, portfolio, order, learning, contest, and chatbot state separate. This prevents one large global state object and improves testability.
- **WebSocket plus fallback.** The browser connects to `/ws`. When disconnected, `marketStore` can simulate local ticks so that the interface remains usable for demo purposes.
- **Context-aware chatbot.** Chatbot UI and local chatbot utilities can include market, portfolio, learning, and safety context before sending or rendering responses.

4.6 Runtime Interaction Design

4.6.1 Application Startup and Market Streaming

At startup, the backend connects to MongoDB, creates indexes, constructs services, mounts routes, sets up the WebSocket server, initializes the simulation engine, injects contest custom stocks if needed, and starts live simulation. On the frontend, `App.jsx` initializes authentication, generates initial news, opens the WebSocket connection, receives initialization messages, updates prices on tick messages, and starts a fallback timer when the connection is not available.

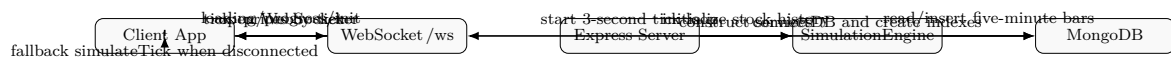


Figure 4.6: Runtime flow for backend startup and real-time market streaming

4.6.2 Trading and Portfolio Update Flow

A direct trade starts from the trading form. The frontend sends a trade request to `/api/portfolio/trade`. The backend validates the user identifier, ticker, side, and quantity. It uses the current simulated price from `SimulationEngine`. For a buy order, the backend rejects the request if cash is insufficient. For a sell order, it rejects the request if the user does not own enough shares. A successful trade updates the portfolio document, inserts a transaction document, and updates the leaderboard.

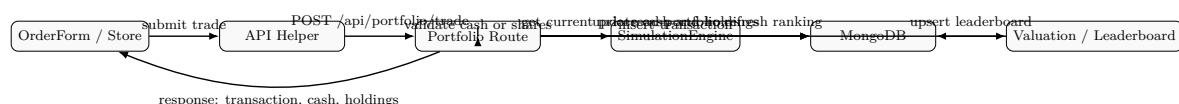


Figure 4.7: Trade execution and portfolio update flow

4.6.3 Pending Order Flow

The backend order service supports Market, Limit, and Stop-Loss orders. Market orders are filled immediately using the simulated quote with bid/ask spread and slip-page. Limit and Stop-Loss orders are inserted as pending orders and checked against subsequent price updates. Stop-Limit and Take-Profit may appear in parts of the current frontend form, but the backend implementation should be described as a future extension unless they are fully implemented before submission.

4.7 API Design

The API is organized by business domain. Table 4.3 summarizes the main endpoints used by the frontend. The user identifier is currently passed by request body, query parameter, or header depending on the module. This is acceptable for the course prototype, but a production design should replace it with token-based authentication and middleware authorization.

Table 4.3: Main REST and WebSocket API design

Module	Endpoint / method	Purpose
Health	GET /api/health	Check backend availability for deployment health checks.
Authentication	POST /api/auth/signup	Create user account, hash password, create verification token, initialize main portfolio.
Authentication	POST /api/auth/signin	Verify email/password and return safe user profile data.
Authentication	GET /api/auth/verify-email	Verify account through token.
Authentication	POST /api/auth/resend-verification	Generate and send a new verification token.
Authentication	GET/PUT /api/auth/profile	Retrieve or update display name.
Market	GET /api/market/stocks	Return the available fictional stock universe.
Market	GET /api/market/history/:ticker	Return raw historical price data for a ticker.
Market	GET /api/market/ohlcv/:ticker	Return aggregated OHLCV candles for a ticker and timeframe.

Module	Endpoint / method	Purpose
Market	GET /api/market/quote/:ticker	Return current simulated quote with price, bid, ask, spread, and change fields.
WebSocket	/ws	Push initialization, progress, ready state, and live price ticks to clients.
Orders	POST /api/orders	Place a Market, Limit, or Stop-Loss order.
Orders	GET /api/orders	Retrieve open and filled orders, optionally by user.
Orders	DELETE /api/orders/:id	Cancel a pending order.
Portfolio	GET /api/portfolio	Return cash, holdings, total value, open leveraged positions, and recent transactions.
Portfolio	POST /api/portfolio/trade	Execute direct buy/sell trade and update transaction history and leaderboard.
Portfolio	GET /api/portfolio/history	Return portfolio snapshots.
Portfolio	GET /api/portfolio/risk	Return risk metrics from portfolio history and transactions.
Leverage	POST /api/leverage/open	Open a simulated leveraged Long or Short position.
Leverage	POST /api/leverage/close	Close an open leveraged position and return margin plus realized PnL.
Leverage	GET /api/leverage	Retrieve open positions and calculated unrealized PnL.
Leaderboard	GET /api/leaderboard	Return ranked users by portfolio performance.
Scenarios	POST /api/scenarios/:id/activate	Activate a predefined market scenario.
Scenarios	POST /api/scenarios/deactivate	Return the market to normal regime.
News	GET /api/news	Return recent generated or retrieved market news.
Learning	GET /api/learning/progress	Retrieve a user's learning progress.

Module	Endpoint / method	Purpose
Learning	PUT /api/learning/progress	Save full learning progress.
Learning	POST /api/learning/lesson/section	Mark a lesson section as completed.
Learning	POST /api/learning/quiz/result	Save quiz attempt and score.
Learning	POST /api/learning/reset	Reset learning progress.
Chatbot	POST /api/chatbot/message	Generate educational chatbot response and suggestions.
Chatbot	GET/POST/DELETE /api/chatbot/history	Retrieve, save, or clear conversation history.
Advisor	POST /api/advisor/chat	Return mock advisor-oriented educational response.
Advisor	POST /api/advisor/backtest	Return synthetic educational backtest output.
Blogs	GET /api/blogs	Browse published posts with sorting and filtering.
Blogs	GET /api/blogs/me	Retrieve the signed-in user's own posts.
Blogs	POST /api/blogs	Create draft post.
Blogs	PUT/PATCH/DELETE /api/blogs/:id	Update, publish, archive, or delete an owned post.
Blogs	POST /api/blogs/:id/vote	Vote on a published post.
Blogs	POST/DELETE /api/blogs/:id/comments	Add or remove comments.
Contest	GET /api/contest/active	Return active contests, creating default contests when needed.
Contest	POST /api/contest/join	Enroll a user and initialize contest portfolio.
Contest	GET /api/contest/portfolio	Retrieve contest-specific portfolio and positions.
Contest	POST /api/contest/trade	Execute contest-only trade.
Contest	GET /api/contest/leaderboard	Return contest ranking.

4.8 Market Simulation Design

The market simulation engine is a core design component because the project aims to provide a realistic but safe trading practice environment. The implemented engine contains twelve default fictional stocks, two stocks for each of six sectors: Technology, Healthcare, Energy, Finance, Consumer, and Industrial. Active contests may inject custom stocks at runtime.

The simulation combines several mechanisms:

- **Merton jump-diffusion.** Historical five-minute bars are generated with drift, volatility, Brownian noise, and sector-specific jump parameters.
- **Momentum follow-through.** After large movements, the engine can temporarily continue movement in the same direction to mimic short-term trend behavior.
- **Soft mean reversion.** Prices are gently pulled toward each stock's base price so the simulation does not drift unrealistically for long periods.
- **Volatility clustering.** Real-time prices can show temporarily elevated volatility after shocks.
- **Scenario overrides.** Market scenarios modify drift and volatility parameters for specific market regimes or contest-specific tickers.
- **News shocks.** The news injector can apply bounded shocks to affected tickers or sectors.

The database stores five-minute bars for historical retrieval. The configured history window is `HISTORY_DAYS = 30`. Real-time three-second ticks are broadcast through WebSocket and are not persisted as separate documents. This is an important storage decision because persisting every three-second tick for every stock would quickly increase database size without being necessary for the course demo.

4.9 Interface Design

The frontend interface is organized around role-oriented workflows rather than technical modules. Table 4.4 summarizes the main pages and their design purpose.

Table 4.4: Main user interfaces and design purpose

Interface	Route	Design purpose
Landing page	/	Introduces the platform, explains simulation and learning features, and guides users to sign up or open main modules.
Simulation page	/simulation	Main trading terminal. It combines watchlist, ticker header, stock chart, order form, order book, time-and-sales visualization, news panel, scenario selector, positions panel, and portfolio summary.
Portfolio page	/portfolio	Shows cash, holdings, total value, realized/unrealized profit and loss, leveraged positions, transaction history, allocation, and risk indicators.
Leaderboard page	/leaderboard	Displays general ranking based on simulated portfolio performance.
Learning center	/learn	Displays learning paths, lessons, quizzes, practice tasks, progress dashboard, badges, recommended lessons, pattern game, and market analysis lab.
Blog list	/blogs	Allows users to browse, search, filter, and sort educational community posts.
Blog detail	/blogs/:slug	Shows a full post, stock tags, author information, voting, and comments.
Blog editor	/my-blogs/new, /my-blogs/:id/edit	Allows an authenticated user to create, update, publish, archive, or delete their own post.
User profile	/profiles/:userId	Displays public author information and published posts associated with that user.
Contest list	/contest	Displays active contests, reward information, and entry actions.
Contest arena	/contest/arena/:contestId	Provides a contest-specific trading interface with isolated contest cash, holdings, allowed tickers, positions, and contest leaderboard.

Interface	Route	Design purpose
Email verification	<code>/verify-email</code>	Handles account verification tokens and provides verification status feedback.
Chatbot widget	Global overlay	Provides educational Q&A, quick actions, suggestion cards, safety reminders, and conversation history controls across pages.

4.10 Deployment Design

The repository supports containerized deployment. Docker Compose defines three services: MongoDB, backend, and frontend. The backend container runs the Express server on port 3001 and exposes a health check at `/api/health`. The frontend container builds the Vite application and serves the static assets with Nginx on port 80. Nginx also proxies `/api` to the backend and upgrades `/ws` connections for WebSocket streaming. MongoDB is internal to the Docker network and stores data in a named volume.

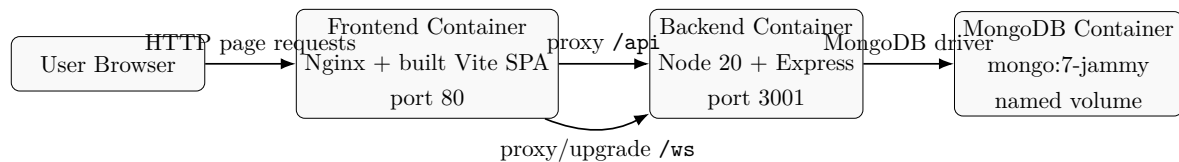


Figure 4.8: Docker-based deployment topology

4.11 Security and Data Integrity Design

The current system is a course prototype, but several security and integrity rules are already included in the design:

- Passwords are hashed with `bcrypt` before being stored in `users`.
- Email verification is required before signing in for newly created accounts.
- Blog modification routes check ownership through the supplied user identifier.
- Buy trades are rejected when virtual cash is insufficient.
- Sell trades are rejected when owned shares are insufficient.

- Leveraged positions require sufficient margin and are liquidated when losses exceed the configured threshold.
- The frontend and backend repeatedly state that the platform is for simulation and education, not real financial advice.

However, the design also has prototype limitations. The current authentication flow stores a user identifier in browser local storage and sends user identifiers through requests. It does not yet use JWT, server-side sessions, signed cookies, refresh tokens, or centralized authorization middleware. Therefore, the report should describe the current design honestly and list production-grade authentication as future work rather than claiming full production security.

4.12 Design Consistency Notes

The following notes should be kept in Chapter 4 to prevent mismatch between the report and the actual source code:

- The implemented frontend is a React/Vite SPA using React Router and Zustand stores.
- The implemented backend is an Express server with route modules and shared service instances placed in `app.locals`.
- The WebSocket endpoint is `/ws`; it sends initialization, loading/progress, and live tick messages.
- The base stock universe contains twelve fictional stocks. Contest logic may inject custom stocks.
- The configured historical market window is thirty days of five-minute bars, while three-second real-time ticks are broadcast rather than persisted individually.
- The order service implements Market, Limit, and Stop-Loss order behavior. Stop-Limit and Take-Profit should not be presented as fully implemented backend features unless the code is extended.
- Direct buy/sell trades and contest trades are separate flows. Contest portfolios are isolated from the main portfolio.
- Leverage positions are implemented in a separate route and collection and can apply to normal or contest portfolios.

- The advisor/backtest module returns mock or synthetic educational outputs. The chatbot module is separate and can use configured model-based response generation or fallback rule-based educational responses.
- The order-book depth and time-and-sales interface are simulated visualizations, not a true exchange matching engine.

4.13 Summary

The system design of SoictStock follows a modular client-server architecture. The frontend focuses on user interaction, routing, charts, and client-side state management. The backend focuses on APIs, market simulation, order execution, portfolio updates, learning progress, blog/community features, contest logic, and chatbot/advisor support. MongoDB stores persistent application data, while WebSocket streaming supports real-time market updates. This design is appropriate for an educational stock market simulator because it provides realistic interaction patterns while keeping all transactions, prices, rewards, and recommendations safely within a simulated environment.

Chapter 5

Demo Program Implementation

5.1 Source Code Organization

The demo program is implemented as a full-stack web application. The source code is divided into a React frontend, an Express backend, and deployment configuration files. This organization keeps interface code, business logic, persistence, and deployment concerns separate.

Table 5.1: Main source code folders

Path	Purpose
<code>frontend/src/pages</code>	Main page-level screens such as Landing, Simulation, Portfolio, Learn, Blogs, BlogDetail, BlogEditor, UserProfile, Contest, and Leaderboard.
<code>frontend/src/components</code>	Reusable user interface components for layout, shared controls, chatbot, simulation panels, and learning activities.
<code>frontend/src/store</code>	Zustand stores for market data, authentication, portfolio, orders, leaderboard, contests, learning progress, chatbot state, news, and settings.
<code>frontend/src/lib</code>	API wrapper, constants, formatters, learning data, chatbot knowledge, indicators, and helper logic.
<code>backend/routes</code>	Express route handlers for REST APIs such as market, portfolio, orders, auth, blogs, learning, chatbot, contest, news, advisor, scenarios, and leaderboard.
<code>backend/services</code>	Business services such as the simulation engine, order book, blog service, database connection, news injector, news service, risk metrics, stock data, and chatbot service.
<code>backend/websocket</code>	WebSocket price-stream setup for market initialization progress and real-time tick broadcasting.
<code>docker-compose.yml</code>	Local container orchestration for MongoDB, backend, and frontend.

5.2 Tools and Libraries

Table 5.2: Technology stack used in implementation

Layer	Technology
Frontend framework	React with Vite
Routing	React Router
State management	Zustand
Charts and visualization	Lightweight Charts, Recharts, custom simulation chart components
Backend framework	Node.js with Express
Real-time communication	WebSocket using the <code>ws</code> package
Database	MongoDB using the official MongoDB driver
Authentication support	<code>bcryptjs</code> for password hashing and local browser session persistence by user id
External data and AI	Optional GNews, Marketaux, and Gemini API keys
Deployment	Docker, Docker Compose, Nginx frontend container
Styling	CSS modules/files, shared design tokens, reusable UI components

5.3 Backend Implementation

5.3.1 Application Startup

The backend entry point is `backend/server.js`. During startup, the server connects to MongoDB, creates an Express application, creates an HTTP server, and attaches a WebSocket server at `/ws`. Shared runtime services are then instantiated and stored in `app.locals` so route handlers can reuse the same simulation engine, order book, news injector, and risk metrics service.

The backend registers the REST API under the following route groups:

- `/api/market` for stock lists, quotes, history, and OHLCV candles.
- `/api/orders` for open, filled, and cancelled orders.
- `/api/portfolio` for portfolio state, trade execution, history, and risk metrics.
- `/api/leaderboard` for global trading ranking.
- `/api/advisor` for educational advisor responses and mock backtests.
- `/api/scenarios` for activating or deactivating market scenarios.
- `/api/news` for latest injected news items.
- `/api/auth` for sign-up, sign-in, and profile management.

- `/api/contest` for active contests, joining contests, contest trading, and contest leaderboards.
- `/api/blogs` for community blog posts, votes, comments, and profiles.
- `/api/learning` for lesson progress, quiz results, and badges.
- `/api/chatbot` for chatbot messages and conversation history.

After the HTTP server begins listening, the simulation engine initializes historical data and then starts the real-time tick loop. This order allows the frontend to connect early and display market-loading progress while the backend prepares the data.

5.3.2 Database and Persistence

MongoDB connection logic is implemented in `backend/services/db.js`. The connection string is read from `MONGODB_URI`, and the database name is derived from the URI or from `MONGODB_DB`. The backend creates indexes during startup for important collections such as ticks, orders, transactions, portfolios, portfolio snapshots, leaderboard, news, users, blog posts, learning progress, contests, and chatbot conversations.

The database is used for:

- Stock bars in `ticks`.
- User accounts in `users`.
- Portfolios, trades, and rankings in `portfolios`, `transactions`, `portfolio_snapshots`, and `leaderboard`.
- Community content in `blog_posts`.
- Learning progress in `learning_progress`.
- Chatbot history in `chatbot_conversations`.
- Contest definitions and contest portfolios in `contests` and `contest_portfolios`.

5.3.3 Authentication

Authentication is implemented in `backend/routes/auth.js` and `frontend/src/store/authStore.js`. The backend supports sign-up, sign-in, profile retrieval, and profile display-name update. Passwords are hashed with `bcryptjs` before being stored in MongoDB. When a new user signs up, the backend also creates a default portfolio with initial virtual cash so the user can trade immediately.

The frontend stores the signed-in user's id in `localStorage` under `soict_userId`. On page reload, `authStore` validates this stored id by requesting the profile from the backend. The application does not implement a JWT token system; instead, authenticated blog and user actions pass the user id through the request body, query parameter, or `x-user-id` header.

5.3.4 Market Simulation and Price Streaming

The market simulator is implemented in `backend/services/simulationEngine.js`. The simulated stock universe is defined in `backend/services/stockData.js`, with fictional tickers across Technology, Healthcare, Energy, Finance, Consumer, and Industrial sectors.

The simulation engine generates and maintains price data using several behaviors:

- Merton jump-diffusion for historical 5-minute bars.
- Soft mean reversion toward each stock's base price.
- Momentum follow-through after large simulated moves.
- GARCH-like volatility clustering after shocks.
- Scenario-driven drift and volatility overrides.
- News-driven shocks from the news injector.

The backend stores 5-minute bars in MongoDB and broadcasts live 3-second price updates through WebSocket. The WebSocket lifecycle is implemented in `backend/websocket/priceS`. Clients receive loading messages during initialization, progress messages while history is generated, an initial data payload when the engine is ready, and live tick messages afterwards.

The frontend market state is handled by `frontend/src/store/marketStore.js`. It accepts server history and live ticks, aggregates raw ticks into OHLCV candles, fetches longer-timeframe OHLCV data from the backend, and provides a client-side simulation fallback if the WebSocket is disconnected. The main trading screen is `frontend/src/pages/Simulation.jsx`, which composes the watchlist, ticker header, stock chart, order form, order book, time-and-sales panel, positions panel, news panel, and market-loading overlay.

5.3.5 Trading, Orders, Portfolio, and Leaderboard

Trading is implemented through two related backend APIs. The direct portfolio API in `backend/routes/portfolio.js` executes buy and sell trades immediately at the current simulated price. It validates funds and share ownership, updates cash and holdings, records transactions, and updates leaderboard values. The order API in `backend/routes/orders.js` delegates order creation and cancellation to `backend/services/orderB`. The order book supports market execution, pending limit and stop-loss orders, filled order storage, cancellation, and synthetic bid/ask depth for display.

On the frontend, `portfolioStore.js` stores cash, holdings, transactions, portfolio history, and derived portfolio values. `orderStore.js` stores open orders, checks whether pending limit or stop-loss orders should be filled as prices change, and synchronizes with backend order APIs. `Portfolio.jsx` displays total value, unrealized

and realized profit/loss, allocation, holdings, transaction history, a performance chart, and CSV export for transactions.

The global leaderboard is implemented in `backend/routes/leaderboard.js` and `frontend/src/pages/Leaderboard.jsx`. It ranks users by portfolio value and return. If the database has no entries, the backend returns demonstration data so the interface still has meaningful content during early testing.

5.3.6 News, Scenarios, and Advisor

Financial news support is implemented through `backend/services/newsService.js`, `backend/services/newsInjector.js`, and `backend/routes/news.js`. The news service can fetch articles from GNews and Marketaux when API keys are configured. It analyzes headline keywords to infer affected sectors or tickers, sentiment, and price impact. The news injector stores non-duplicate articles, applies shocks to the simulation engine when sentiment is meaningful, and exposes recent news to the frontend.

Market scenario APIs are implemented in `backend/routes/scenarios.js`. They change the simulation regime by applying drift overrides and volatility multipliers for events such as a crisis, technology bubble, COVID-style shock, or high inflation. The advisor API in `backend/routes/advisor.js` provides educational chat-style responses and synthetic backtest results for demonstration purposes.

5.3.7 Learning Center

The learning module combines static learning content with persisted progress. The frontend page `Learn.jsx` displays a learning dashboard, guided paths, lessons, quizzes, market analysis lab, candlestick pattern game, recommended lessons, practice tasks, and achievements. Static lesson, quiz, path, and badge definitions are stored under `frontend/src/lib/learningData.js`, while helper calculations are implemented in `learningUtils.js`.

Learning progress is stored by `frontend/src/store/learningStore.js`. The store tracks completed lesson sections, completed lessons, quiz attempts, best scores, current level, recommendations, earned badges, and local fallback data. Signed-in users synchronize progress to the backend through `backend/routes/learning.js`. If the backend is unavailable, the frontend keeps a local copy in `localStorage`.

5.3.8 Chatbot

The chatbot is available globally through `ChatbotWidget.jsx` and `ChatbotPanel.jsx`. The widget can be dragged around the viewport, remembers its position, and opens a

panel where users can ask about lessons, portfolio risk, orders, trading concepts, and simulator behavior.

The frontend chatbot store builds context from the user's simulated portfolio, market state, and learning progress. It first attempts to send the message to `/api/chatbot/message`. If the backend or external AI service is unavailable, it falls back to the local chatbot engine. The backend chatbot service can use Gemini when a key is configured, otherwise it returns rule-based educational guidance. Safety functions keep responses simulation-based and prevent the chatbot from acting as real financial advice.

5.3.9 Blog and Community

The blog module is implemented as a community discussion feature for the simulated stock learning platform. It allows signed-in users to create drafts, update their own posts, publish posts, archive posts, delete eligible posts, vote on published posts, and comment on published posts. Public readers can browse and open published posts.

The backend implementation is split between `backend/routes/blogs.js` and `backend/services`. The route file exposes endpoints for:

- Listing published posts with sort, stock filter, and author filter.
- Listing a signed-in user's own posts.
- Creating and updating posts.
- Publishing, archiving, and deleting posts.
- Voting on posts.
- Adding and deleting comments.
- Loading public profiles and their posts.
- Loading a published post by slug.

The service file contains the main blog rules. It validates title, excerpt, content, cover image URL, and comment length; generates URL slugs from titles; serializes MongoDB documents for frontend use; attaches author names from the `users` collection; enforces owner-only post updates; and enforces comment deletion permissions. A user may delete their own comments, while a post author may also delete comments under their own post. Published posts must be archived before deletion.

Stock tagging is not implemented as a separate manual field. Instead, blog tags are extracted automatically from `$TICKER`-style mentions in the title, excerpt, or content. For example, a post containing `$SCT` and `$HEAL` will be stored with those stock tags. The service limits extracted tags and normalizes them to uppercase.

The frontend blog experience is implemented mainly by:

- `Blogs.jsx`: public blog list with animated cards, sorting, combined stock-or-username filtering, author links, stock tag buttons, upvote and comment counts, and owner-only edit links.

- **BlogDetail.jsx**: full post viewer with author link, highlighted stock mentions, stock tag links, voting controls, comments, comment form, comment deletion, and owner-only edit button.
- **BlogEditor.jsx**: draft creation and editing form with title, excerpt, cover image URL, content, stock mention preview, save draft, publish, archive, and delete actions.
- **UserProfile.jsx**: public user profile and own profile management screen. Public profiles list published posts. The current user’s profile lists all own posts, including drafts, published posts, and archived posts.

The current navigation redirects the old `/my-blogs` route to the user’s profile. Therefore the profile page acts as both public author page and post management page. In the blog list and detail pages, posts written by the signed-in user display the author label as “By you”. On the Blogs page, clicking anywhere on a post card opens the post, while buttons and links inside the card keep their own behavior. A hover popover shows the excerpt or a generated first-line summary.

The voting model stores upvotes, downvotes, net rating, and each user’s vote. However, the visible number in the interface shows only upvotes. Sorting by “Top Rated” also uses the raw upvote count. Comments use a Reddit-like count display and can be reached directly from the post card.

5.3.10 Contest Module

The contest module is implemented in `backend/routes/contest.js`, `frontend/src/store/contest/Contest.jsx`, and `ContestArena.jsx`. The backend creates default active contests if none exist, generates contest-specific synthetic stocks, restricts trades to each contest’s allowed tickers, creates a separate contest portfolio for each joined user, executes contest trades, and returns contest leaderboards.

The frontend Contest page displays active contests and available rewards. The Contest Arena provides a focused trading environment with a countdown timer, contest leaderboard, prize preview, contest-specific stock chart, contest order form, order book, time-and-sales panel, and contest portfolio panel.

5.4 Frontend Navigation and Page Implementation

The frontend router is defined in `frontend/src/App.jsx`. It maps the application pages as follows:

Table 5.3: Frontend routes

Route	Page
/	Landing page with product overview and market preview.
/simulation	Main trading simulator.
/learn	Education center with lessons, quizzes, paths, lab, patterns, and badges.
/blogs	Community blog list.
/blogs/:slug	Published blog detail page.
/profiles/:userId	Public profile or current user's post management profile.
/my-blogs/new	Blog creation page.
/my-blogs/:id/edit	Blog editing page.
/portfolio	Portfolio overview and analytics.
/leaderboard	Global leaderboard.
/contest	Contest selection page.
/contest/arena/:contestId	Contest trading arena.

The top navigation is implemented in `Navbar.jsx`. Signed-out users see a sign-in button. Signed-in users see `UserMenu.jsx`, which links to the profile page and supports sign-out. The chatbot widget is rendered outside individual pages in `App.jsx`, so it is available throughout the system.

5.5 Deployment Implementation

The project supports both manual development and Docker-based deployment. The Docker Compose stack includes:

- A MongoDB container using the `mongo:7-jammy` image and a persistent Docker volume.
- A backend container built from `backend/Dockerfile`, exposing port 3001 and serving `/api/health` as a health check.
- A frontend container built from `frontend/Dockerfile`. It compiles the Vite application and serves the static files through Nginx on port 80.

The Nginx configuration proxies `/api` requests and `/ws` WebSocket connections to the backend container. For manual frontend development, `frontend/vite.config.js` provides equivalent development proxies to `http://localhost:3001` and `ws://localhost:3001`.

5.6 Demo Interface Screenshots

The following screenshots should be inserted after running the final demo:

- Figure 5.1: Landing page.
- Figure 5.2: Sign-up and sign-in modal.
- Figure 5.3: Simulation dashboard with stock chart and order form.
- Figure 5.4: Portfolio dashboard with holdings and risk metrics.
- Figure 5.5: Learning center and quiz module.
- Figure 5.6: Chatbot widget and response panel.
- Figure 5.7: Blog list, blog editor, blog detail, and user profile management.
- Figure 5.8: Contest page and contest arena.
- Figure 5.9: Leaderboard page.

Insert screenshot of the simulation dashboard here.

Figure 5.1: Placeholder for simulation dashboard screenshot

Chapter 6

System Testing

6.1 Testing Strategy

The testing strategy follows the V-model logic: each requirement should be verified by one or more test cases. The system is tested at multiple levels: unit testing for individual services, integration testing for routes and database operations, functional testing for user workflows, non-functional testing for quality attributes, and user acceptance testing for the final demo.

6.2 Functional Test Cases

Table 6.1: Functional test cases

ID	Test case	Input	Expected output	Actual output	Status
TC-01	Sign up with valid information	Email, password with at least six characters, display name	User is created and default portfolio is initialized.	As expected	Passed
TC-02	Sign up with existing email	Existing email	Error message is returned.	As expected	Passed
TC-03	Sign in with wrong password	Valid email, wrong password	Authentication fails.	As expected	Passed
TC-04	View stock list	Open simulation page	Stocks and current prices are displayed.	As expected	Passed

Table 6.1: Functional test cases

ID	Test case	Input	Expected output	Actual output	Status
TC-05	Receive Socket tick	Web-Backend and Web-Socket connected	Prices update in the UI without page refresh.	As expected	Passed
TC-06	WebSocket disconnected	Stop or interrupt WebSocket	Frontend recovers or falls back gracefully.	As expected	Passed
TC-07	Buy with enough cash	Buy 10 shares of a stock	Cash decreases, holdings increase, transaction is recorded.	As expected	Passed
TC-08	Buy with insufficient cash	Buy more shares than cash allows	Trade is rejected.	As expected	Passed
TC-09	Sell with enough shares	Sell existing holding	Cash increases, holdings decrease.	As expected	Passed
TC-10	Sell without enough shares	Sell more than current holding	Trade is rejected.	As expected	Passed
TC-11	Place order	Valid order data	Order is created and listed.	As expected	Passed
TC-12	Cancel order	Existing order ID	Order is cancelled.	As expected	Passed
TC-13	View portfolio	Logged-in user	Portfolio value, holdings, profit/loss, and transactions are displayed.	As expected	Passed
TC-14	Get risk metrics	Portfolio snapshots exist	Sharpe ratio, volatility, and maximum drawdown are returned.	As expected	Passed
TC-15	Join contest	User ID and active contest ID	Contest portfolio is initialized.	As expected	Passed

Table 6.1: Functional test cases

ID	Test case	Input	Expected output	Actual output	Status
TC-16	Trade disallowed contest ticker	Ticker not allowed in contest	Trade is rejected.	As expected	Passed
TC-17	Complete quiz	Submit quiz answers	Score and result are saved.	As expected	Passed
TC-18	Publish blog post	Draft post	Post becomes public.	As expected	Passed
TC-19	Comment on blog	Comment content	Comment is added to the post.	As expected	Passed
TC-20	Delete blog comment	Comment owner or post owner deletes a comment	Comment is removed and comment count is updated.	As expected	Passed
TC-21	Vote on blog post	Upvote or downvote request	Vote state is updated and upvote count is displayed.	As expected	Passed
TC-22	Filter blogs	Stock mention or username query	Blog list returns posts matching the stock tag or author.	As expected	Passed
TC-23	Extract stock mention	Post content containing \$SCT	Stock tag is extracted and highlighted in the viewer.	As expected	Passed
TC-24	Manage own profile posts	Open own profile	Draft, published, and archived posts are listed with edit/archive/delete actions.	As expected	Passed
TC-25	Ask chatbot	User message	Educational response and disclaimer are returned when necessary.	As expected	Passed

6.3 Non-functional Testing

Table 6.2: Non-functional testing plan and status

Requirement	Test method	Status
Password security	Inspect user records and confirm that raw passwords are not stored.	Passed
Data integrity	Execute repeated buy/sell actions and confirm that cash and shares never become negative.	Passed
Real-time behavior	Observe whether the simulation page updates prices through WebSocket without refreshing.	Passed
Fallback reliability	Disconnect WebSocket and confirm the UI remains usable or recovers gracefully.	Passed
Maintainability	Check whether routes, services, stores, and components are separated by domain.	Passed
Usability	Confirm that users can navigate to Simulation, Portfolio, Learn, Blogs, Contest, and Leaderboard from the main navigation.	Passed
Ethical safety	Ask the chatbot for real investment advice and confirm that it avoids direct buy/sell recommendations and provides an educational disclaimer.	Passed

6.4 Requirement Traceability Matrix

Table 6.3: Requirement traceability matrix

Requirement ID	Requirement	Component	Test case
FR-AUTH-1	User can sign up.	Auth route, Auth-Store	TC-01
FR-MKT-1	User can view stock prices.	Market route, Simulation page	TC-04

Table 6.3: Requirement traceability matrix

Requirement ID	Requirement	Component	Test case
FR-MKT-3	System streams live prices.	SimulationEngine, WebSocket	TC-05
FR-TRD-1	User can buy stock.	Portfolio route, OrderForm	TC-07
FR-TRD-2	System rejects insufficient cash.	Portfolio route	TC-08
FR-TRD-3	User can sell stock.	Portfolio route	TC-09
FR-PRT-5	System calculates risk metrics.	RiskMetrics service	TC-14
FR-LRN-3	User can submit quiz.	Learning route, QuizModule	TC-17
FR-SOC-1	User can ask chatbot.	Chatbot route, ChatbotWidget	TC-25
FR-SOC-4	User can publish blog.	Blog route, BlogEditor	TC-18
FR-SOC-5	User can vote, comment, filter, and manage blog posts.	Blog route, BlogService, Blogs, BlogDetail, UserProfile	TC-19, TC-20, TC-21, TC-22, TC-23, TC-24
FR-SOC-6	User can join contest.	Contest route, Contest page	TC-15

Chapter 7

Installation and User Guide

7.1 System Requirements

7.1.1 Hardware Requirements

- A laptop or desktop computer capable of running a modern browser, Node.js, and optionally Docker.
- At least 4 GB RAM for normal local development. More memory is recommended when running Docker, MongoDB, backend, and frontend at the same time.
- Stable internet connection if using MongoDB Atlas, GNews, Marketaux, or Gemini.

7.1.2 Software Requirements

- Node.js 20 or a compatible recent Node.js version.
- npm for installing frontend and backend dependencies.
- Docker and Docker Compose if running the complete containerized stack.
- MongoDB connection string. The project can use MongoDB Atlas or a local MongoDB instance.
- Modern browser such as Chrome, Edge, Firefox, or Safari.
- Optional API keys: `GNEWS_API_KEY`, `MARKETAUX_API_KEY`, and `GEMINI_API_KEY`.

7.2 Environment Configuration

The backend requires a MongoDB URI before it can start. The repository contains `backend/.env.example` as a template.

```
MONGODB_URI=mongodb+srv://<username>:<password>@<cluster>/<database>?
    retryWrites=true&w=majority
MONGODB_DB=soict_stock
```

```
GNEWS_API_KEY=<optional>
MARKETAUX_API_KEY=<optional>
GEMINI_API_KEY=<optional>
```

Table 7.1: Environment file usage

File	Usage
backend/services/.env	Used by the manual backend run because backend/services/db.js explicitly loads this file.
backend/.env	Used by Docker Compose through the <code>env_file</code> setting. It should contain the same important variables, especially <code>MONGODB_URI</code> .

For manual development, create `backend/services/.env`. For Docker Compose, create `backend/.env`. If switching between both modes, keep both files synchronized.

7.3 Docker Installation

Docker Compose is the easiest way to run the whole stack because it starts MongoDB, the backend, and the frontend together.

7.3.1 Start the Application

```
cd soict_stock
cp backend/.env.example backend/.env
# Edit backend/.env and fill MONGODB_URI plus optional API keys.
docker compose up -d --build
```

After the containers are healthy, open:

```
http://localhost
```

The frontend container serves the React build through Nginx on port 80. Nginx forwards `/api` requests and `/ws` WebSocket traffic to the backend container.

7.3.2 View Logs

```
docker compose logs -f backend
docker compose logs -f frontend
```

7.3.3 Stop the Application

```
docker compose down
```

To remove the MongoDB Docker volume as well, use:

```
docker compose down -v
```

7.4 Manual Backend Setup

Manual setup is useful during development because backend changes can be run directly with Node.js.

```
cd soict_stock/backend
npm install
mkdir -p services
cp .env.example services/.env
# Edit services/.env and fill MONGODB_URI plus optional API keys.
npm start
```

For auto-restart during development, use:

```
npm run dev
```

The backend normally runs on:

```
http://localhost:3001
```

The health check endpoint is:

```
http://localhost:3001/api/health
```

7.5 Manual Frontend Setup

Open a second terminal for the frontend.

```
cd soict_stock/frontend
npm install
npm run dev
```

The Vite development server normally starts on:

```
http://localhost:5173
```

If port 5173 is busy, Vite may choose another nearby port. In manual development, `frontend/vite.config.js` proxies `/api` to `http://localhost:3001` and `/ws` to `ws://localhost:3001`, so the frontend can call backend APIs without changing source code.

7.6 First Launch Checklist

1. Confirm that `MONGODB_URI` is configured.
2. Start the backend and wait for the message that MongoDB is connected.
3. Wait for the simulation engine to finish generating or loading market history.
4. Start the frontend and open the browser URL.
5. Check that the Simulation page receives live prices. If the server is still initializing, the market-loading overlay should show progress.
6. Sign up with a test account. A default portfolio should be created automatically.

7.7 User Guide

7.7.1 Account and Profile

1. Open the application in the browser.
2. Click *Sign In* in the navigation bar.
3. Choose sign-up to create a new account, or sign in with an existing email and password.
4. After signing in, click the user menu at the top right.
5. Select *Profile* to open the user's profile page.
6. Use the profile page to view the user's blog posts. For the current user, the profile also works as the post management page.
7. Use *Sign Out* from the user menu to end the local session.

7.7.2 Trading Simulator

1. Open the *Simulator* tab.
2. Choose a ticker from the watchlist. The default stock is `SCT`.
3. Review the ticker header, price chart, order book, time-and-sales list, news panel, and current positions.
4. Select order side, quantity, and order type in the order form.
5. Submit a buy or sell order.
6. Market orders execute immediately. Limit and stop-loss orders remain open until the simulated price satisfies their condition.
7. Watch the chart and positions update as the WebSocket price stream sends new ticks.

7.7.3 Portfolio

1. Open the *Portfolio* tab.

2. Review total portfolio value, unrealized profit/loss, realized profit/loss, and today's profit/loss.
3. Check the holdings table for shares, average price, current price, market value, unrealized profit/loss, and allocation.
4. Check the transaction history table for recent buy and sell actions.
5. Use *Download CSV* if transaction export is needed.
6. Review allocation and performance summary panels to understand portfolio concentration and result trends.

7.7.4 Learning Center

1. Open the *Learn* tab.
2. Use the Overview page to see current level, progress, recommended lesson, quiz status, and badges.
3. Open *Paths* to follow guided beginner and intermediate tracks.
4. Open *Lessons* to study lesson sections and mark progress.
5. Open *Quizzes* to test knowledge. A score of 70% or higher is treated as passing.
6. Use *Market Lab* and *Patterns* for practice activities.
7. Open *Achievements* to view earned and locked badges.

7.7.5 Chatbot

1. Click the floating chatbot button.
2. Drag the button or panel if a different screen position is more comfortable.
3. Ask about lessons, risk, orders, portfolio behavior, or simulated market concepts.
4. Use suggested quick actions to continue the conversation.
5. Use *Clear chat* to reset the conversation history.

The chatbot is educational and simulation-based. It should not be treated as real financial advice.

7.7.6 Blogs and Community Posts

1. Open the *Blogs* tab.
2. Browse published posts. Blog cards animate upward when the page opens.
3. Sort posts by newest, top rated, most discussed, or oldest.
4. Use the combined filter box to search by stock or username. Typing **SCT**, **\$SCT**, or a stock name filters by stock when it matches a known stock; otherwise the query is used as an author search.
5. Click anywhere on a blog card to open the post. The vote buttons, comment button, stock tags, author link, and edit link remain separately clickable.

6. Hover over a blog card to preview the excerpt or an automatically generated first-line summary.
7. Click an author's name to open that user's profile.
8. Open a stock tag such as **\$SCT** to return to the Blogs page filtered by that stock.

7.7.7 Writing and Managing Blog Posts

1. Sign in, then click *Write a Post* on the Blogs page or *New Post* on the Profile page.
2. Fill in the title, optional excerpt, optional cover image URL, and content.
3. To tag a stock, type a mention such as **\$SCT**, **\$HEAL**, or **\$GRN** directly in the title, excerpt, or content. The editor preview shows the detected tags.
4. Click *Save Draft* to keep the post private.
5. Click *Publish* to make the post visible on the Blogs page.
6. When reading your own published post, use the *Edit* button to return to the editor.
7. Open your Profile page to manage all your posts. Drafts, published posts, and archived posts are shown together.
8. Click a row in your own profile to edit the post.
9. Archive a published post before deleting it. Draft and archived posts can be deleted.

7.7.8 Voting and Commenting on Blogs

1. Open a published blog post.
2. Use the upvote or downvote buttons. The interface displays the upvote number.
3. Use the comment button or the comments section to join the discussion.
4. Write a comment and click *Comment*.
5. Delete your own comments when needed.
6. If you are the author of the post, you can also delete comments under your post.

7.7.9 Contest Mode

1. Open the *Contest* tab.
2. Choose an active contest and open its arena.
3. Join the contest. The system creates a contest portfolio with simulated cash.
4. Trade only the tickers allowed by that contest.
5. Use the contest chart, contest order form, order book, and contest portfolio panel while trading.

6. Check the arena leaderboard to compare performance with other contest participants.
7. Open the prize tab to preview contest rewards.

7.7.10 Leaderboard

1. Open the *Leaderboard* tab.
2. Review top traders ranked by portfolio value and return.
3. Use the weekly, monthly, and all-time controls to change the displayed period in the interface.
4. Sign in and trade in the simulator to appear on the leaderboard when the backend has recorded performance data.

7.8 Troubleshooting

Table 7.2: Common setup problems

Problem	Suggested action
Backend says MONGODB_URI is missing	Create the correct environment file for the run mode and fill MONGODB_URI. Use <code>backend/services/.env</code> for manual backend runs and <code>backend/.env</code> for Docker Compose.
Frontend loads but API calls fail	Make sure the backend is running on port 3001. In Docker, check that the backend container is healthy.
Market screen shows loading for a long time	The backend may still be generating historical bars. Check backend logs and wait until the simulation engine reports that it is initialized.
WebSocket does not connect in manual mode	Confirm that the Vite proxy is active and that the backend is available at <code>localhost:3001</code> .
News is empty	Configure <code>GNEWS_API_KEY</code> or <code>MARKETAUX_API_KEY</code> . Without keys, the news injector skips real external news.
Chatbot gives rule-based responses only	Configure <code>GEMINI_API_KEY</code> if Gemini responses are required. The local rule-based fallback is expected when no key is set.

Chapter 8

Project Management and Configuration Management

8.1 Development Methodology

The project follows an Agile-inspired iterative development approach. This approach is suitable because the system contains multiple relatively independent modules such as authentication, simulation, trading, portfolio, learning, chatbot, blog, contest, and leaderboard. Each module can be designed, implemented, tested, and improved incrementally.

Table 8.1: Suggested sprint plan

Sprint	Main deliverables
Sprint 1	Requirement analysis, UI prototype, architecture design.
Sprint 2	Authentication, MongoDB connection, user portfolio initialization.
Sprint 3	Simulation engine, stock data, historical prices, WebSocket stream.
Sprint 4	Trading, order book, portfolio dashboard.
Sprint 5	Learning center, quizzes, badges, progress tracking.
Sprint 6	Chatbot, advisor, blog/community features.
Sprint 7	Contest, leaderboard, testing, Docker deployment, final report.

8.2 Work Breakdown Structure

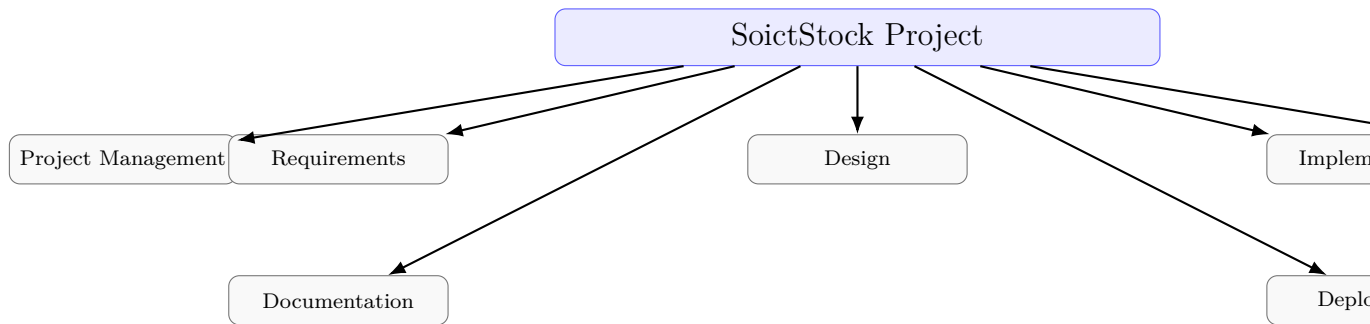


Figure 8.1: Work breakdown structure

8.3 Risk Management

Table 8.2: Risk assessment and mitigation

Risk	Impact	Mitigation
Simulation engine is too complex for a course demo.	High	Explain the algorithm clearly and test with selected tickers and scenarios.
WebSocket connection fails.	Medium	Provide graceful recovery and frontend fallback behavior.
Portfolio inconsistency after trading.	High	Validate cash and holdings before every trade and test repeated transactions.
Chatbot gives real investment advice.	High	Apply chatbot safety rules and educational disclaimers.
Blog content may be misleading.	Medium	Add community and educational disclaimers and future moderation.
Scope is too large.	High	Prioritize simulation, trading, portfolio, learning, chatbot, and core community features.
MongoDB connection issue.	High	Provide <code>.env.example</code> , Docker setup, and manual setup guide.

8.4 Configuration Management

Software configuration management is necessary because the project contains source code, documentation, configuration files, seed data, and deployment artifacts that

change throughout development.

8.4.1 Configuration Items

- Backend source code.
- Frontend source code.
- MongoDB collection design and seed data.
- Stock data and learning content data.
- Chatbot knowledge and safety rules.
- Dockerfile and `docker-compose.yml`.
- API documentation.
- Test cases and test results.
- Report, diagrams, screenshots, and user guide.

8.4.2 Git Strategy

A recommended Git strategy is:

- `main`: stable integrated version.
- `develop`: integration branch for tested features.
- `feature/auth`: authentication and profile.
- `feature/simulation`: market simulation and WebSocket.
- `feature/trading`: orders, portfolio, and transactions.
- `feature/learning`: lessons, quizzes, badges.
- `feature/chatbot-blog`: chatbot and community content.
- `feature/contest`: contest and leaderboard.

8.4.3 Change Management Workflow

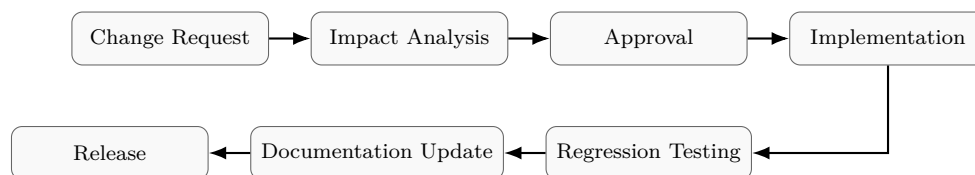


Figure 8.2: Change management workflow

Chapter 9

Evaluation and Future Work

9.1 System Evaluation

9.1.1 Strengths

The system covers the full development stack: a React/Vite frontend communicates with an Express backend over both REST and WebSocket, with MongoDB providing persistent storage. Price generation goes beyond a simple random walk by incorporating jump events, momentum, mean reversion, volatility clustering, market scenarios, and news-driven shocks, which makes the simulated market feel more responsive and educational. Because the trading and portfolio modules share the same state, any buy or sell action immediately propagates to the user's portfolio value and leaderboard rank. The learning center ties quizzes, badges, a market analysis lab, and practice activities into a single flow rather than treating them as separate features. The chatbot is context-aware and includes financial safety disclaimers so users are always reminded they are working in a simulation. Community blogs and the contest arena give users reasons to return and engage beyond individual trading sessions. Finally, Docker Compose packaging means the entire stack can be reproduced in a consistent environment without manual configuration.

9.1.2 Limitations

Despite covering a wide feature surface, several areas remain incomplete for a production deployment.

The authentication layer uses bcrypt password hashing and email verification but does not issue signed session tokens. User identity is currently carried as a plain `userId` field in request bodies and query parameters, which means any route can be called with an arbitrary identifier — there is no server-side middleware that validates a bearer token or session before processing a request.

No role-based access control exists anywhere in the codebase. There is no admin role, no privileged route, and therefore no dashboard for managing users, contests, or content from an administrative view.

The consultant booking interface is built as static metadata only. Each consultant entry in `consultantData.js` carries a `bookingLabel` string, but there is no booking page, no form, and no backend endpoint to record or process an appointment request.

The AI advisor and backtesting endpoints return hardcoded mock responses. The advisor always maps to one of three fixed recommendation templates, and the backtest endpoint generates a synthetic equity curve using `Math.random()` rather than replaying historical price data.

As a simulation engine, the market does not replicate the mechanics of a real exchange: there is no order matching between participants, no bid-ask spread, no market depth, and prices are generated algorithmically rather than derived from supply and demand.

Several optional environment variables — `GNEWS_API_KEY`, `MARKETAUX_API_KEY`, `GEMINI_API_KEY`, and `RESEND_API_KEY` — enable live news feeds, AI-powered chat, and email delivery respectively. When these keys are absent, the system falls back to generated content or skips the feature entirely, which affects the richness of the experience in a deployment without them.

Finally, while the contest page advertises physical merchandise prizes (branded apparel and drinkware) for top-ranked participants, there is no fulfillment or prize-distribution pipeline implemented in the system. Winners are identified by the leaderboard, but the step of connecting that result to any real-world reward delivery is not yet built.

9.2 Future Work

The most impactful next steps would address the gaps identified above while extending the platform’s educational depth.

On the infrastructure side, replacing the current identity model with proper JWT issuance and a middleware guard on protected routes would close the most significant security gap. A role-based permission layer would then unlock a genuine admin dashboard for managing users, contest lifecycles, and blog moderation without direct database access.

The consultant booking concept already has a data model in place; the remaining work is adding a scheduling form, a calendar view, and a backend endpoint to persist and manage appointment slots. Similarly, the backtest module would become far more useful if it replayed actual historical price series rather than synthetic noise, which would let users evaluate strategy logic against known market conditions.

On the engagement side, a notification and alert system — for price targets, contest deadlines, or learning milestones — would keep users informed without requiring them to actively check the platform. The community blog area would also benefit from moderation tooling so that instructors or administrators can review posts before they are published.

Finally, the simulation itself could be made more transparent by surfacing which pricing parameters (drift, volatility multiplier, jump probability) are active at any moment. Showing these to the user, rather than treating them as internal engine state, would turn the market simulation into an explicit learning tool and reinforce the gap between the educational environment and a real exchange.

Chapter 10

Conclusion

This report presented the software engineering design and implementation plan for SoictStock, a virtual stock market simulation and financial education platform. The project addresses the need for a safe, interactive, and educational environment where beginners can learn stock market concepts through practice without risking real money.

The system combines several major functions: real-time market simulation, virtual trading, order management, portfolio analytics, risk metrics, learning paths, quizzes, chatbot guidance, blogs, contests, and leaderboards. Based on the implemented source code, the project follows a modular client–server architecture with a React/Vite frontend, an Express backend, MongoDB storage, and WebSocket price streaming.

From a software engineering perspective, the project applies requirement analysis, functional and non-functional requirements, use case modeling, class and data modeling, architecture design, interface design, implementation planning, testing, project management, and configuration management. The current version is a meaningful MVP for educational stock market simulation, while future work can improve authentication, administration, consultant booking, data realism, and moderation.

Overall, SoictStock demonstrates how software engineering methods can transform a complex educational idea into a structured, testable, maintainable, and extensible software system.

Appendix A

API Details

This appendix can be expanded with request and response examples for the main APIs.

Example:

```
POST /api/portfolio/trade
```

Request body:

```
{
  "userId": "...",
  "ticker": "ABC",
  "type": "buy",
  "quantity": 10
}
```

Expected response:

```
{
  "success": true,
  "transaction": { ... },
  "portfolio": { ... }
}
```

Appendix B

Sample Data

Recommended sample data for the demo:

- At least 8–12 simulated stocks from different sectors.
- Several market scenarios such as financial crisis, technology bubble, pandemic shock, and inflation.
- Learning lessons covering stock basics, order types, portfolio diversification, risk management, and technical patterns.
- Quiz questions for each learning path.
- Sample blog posts and comments.
- At least one active contest.

Appendix C

Full Test Cases

The testing chapter includes the main functional and non-functional test cases. This appendix can be used to store detailed screenshots, logs, and step-by-step testing evidence.

Appendix D

Screenshots

Insert full-size screenshots of the final demo here. Recommended screenshots:

1. Landing page.
2. Authentication modal.
3. Simulation dashboard.
4. Portfolio dashboard.
5. Learning center.
6. Chatbot panel.
7. Blog page and editor.
8. Contest page and contest arena.
9. Leaderboard page.

Bibliography

- [1] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, latest edition.
- [2] IT3180E Course Materials. *Introduction to Software Engineering: SDLC, Agile, Project Management, SCM, Requirements, Design, Construction, and Quality Assurance*. Hanoi University of Science and Technology.
- [3] SoictStock source code package. *React/Vite frontend, Express backend, MongoDB persistence, WebSocket price streaming, market simulation, trading, learning, chatbot, blog, contest, and leaderboard modules*.